

The Interactive MRI k -Space Simulator

A Complete Concept-First Guide

From “What is a frequency?” to a Finished Desktop Application

Theory in plain English · Mathematics explained line-by-line · Implementation as checklists

Companion guide for the 6-week roadmap
(BUET CSE 219 / CSE 220)

August 22, 2026

Abstract

This document teaches you *everything* you need to build the Interactive MRI k -Space Reconstruction Simulator — and, more importantly, to *understand* it deeply enough to answer any question about it. Every chapter follows the same rhythm: first the **concept in plain English** (with analogies), then the **mathematics explained gently**, then a **diagram** so you can *see* the idea, then a **step-by-step implementation checklist** (deliberately without code — the bullets tell you *what* to build, so the code stays yours), and finally **tests and self-check questions**. Nothing in this project is magic. It is all matrix arithmetic, applied patiently.

Contents

1	The Big Picture: What Are We Actually Building?	3
1.1	The whole project, explained in one short story	3
1.2	The five ideas the entire application rests on	4
1.3	The data-flow pipeline, drawn as a picture	4
1.4	The files you will create, and what each one is responsible for	4
2	Milestone 1 — Images, Waves, and the Fourier Transform	7
2.1	Step 1: An image is just a matrix	7
2.2	Step 2: What is a “spatial frequency”?	8
2.3	Step 3: The Fourier transform, gently	9
2.3.1	The continuous version (the “textbook” formula)	9
2.3.2	The discrete version (the one your computer runs)	10
2.3.3	k -space values are complex numbers, and that matters	10
2.4	Step 4: <code>fftshift</code> , or putting the recipe’s center in the middle	10
2.5	Step 5: Why we display $\log(1 + K)$	11
2.6	Step 6: The anatomy of k -space	11
2.7	Step 7: Parseval’s theorem, your built-in correctness test	12
2.8	Implementation guide	12
3	Milestone 2 — Sampling, Undersampling, and Aliasing	15
3.1	Why scanning takes time, and why we are going to cheat	15
3.2	The Nyquist rule in plain English	15
3.3	Sampling masks: undersampling is just a multiplication	16
3.4	The Point Spread Function: predicting the artifact before you see it	18
3.5	Implementation guide	18
4	Milestone 3 — Gibbs Ringing, Apodization, and Noise	21
4.1	The problem: sharp scissors leave ripples	21
4.2	The fix: apodization windows	22
4.3	Thermal noise: why real scans are grainy	23
4.4	Implementation guide	24
5	Milestone 4 — Radial & Spiral Trajectories, Density Compensation, and Grid- ding	27
5.1	Why scanners abandon straight lines	27
5.2	Drawing the paths: it is only trigonometry	28
5.3	The density problem: spokes crowd the center	29
5.4	Gridding: getting scattered points back onto the FFT’s grid	30
5.5	What the artifacts look like now	31
5.6	Implementation guide	31
6	Milestone 5 — Measuring Image Quality: MSE, PSNR, and SSIM	34
6.1	From “it looks worse” to actual numbers	34
6.2	MSE: the average squared mistake	34
6.3	PSNR: MSE on a decibel scale	35
6.4	SSIM: asking what a human would say	35
6.5	The error heatmap: seeing <i>where</i> it went wrong	36
6.6	Implementation guide	36

7	Milestone 6 — The Interactive GUI and Application Architecture	39
7.1	From engine to instrument	39
7.2	The layout	39
7.3	The one problem that ruins naive GUIs: freezing	40
7.4	The architecture that keeps six weeks of code sane	41
7.5	Implementation guide	42
8	The Build Plan — Order of Work, Edge Cases, and Polish	45
8.1	Why we build things in this exact order	45
8.2	Edge cases you must handle (the Week 6 checklist)	46
8.3	When you get stuck: the debugging ladder	47
9	Reference: Glossary and Examiner’s Question Bank	48
9.1	Glossary — every intimidating term, explained in plain words	48
9.2	The examiner’s question bank	50
10	Team Onboarding — Environment Setup and the Project Map	53
10.1	What must already be on your machine	53
10.2	Setup, step by step	53
10.3	The project map	55
11	PySide6 Field Guide — Just Enough Qt, in Plain English	58
11.1	Idea 1: the event loop — your program becomes a waiter	58
11.2	Idea 2: widgets and layouts — Lego bricks that resize themselves	59
11.3	Idea 3: signals and slots — widgets announce, they never command	60
11.4	Embedding matplotlib: the canvas is just another widget	60
11.5	Keeping it responsive: the two-line summary of Chapter 7	61
11.6	Making it look good: the two-step styling recipe	61
11.7	The five classic beginner traps	62

The Big Picture: What Are We Actually Building?

1.1 The whole project, explained in one short story

Let us start with something that surprises almost everyone the first time they hear it. An MRI scanner never takes a photograph. There is no camera inside the machine and no lens pointing at the patient. What the scanner actually does is measure *radio signals*, and those radio signals correspond to the *frequencies* that are hidden inside the slice of the patient's body being examined. A computer then takes those frequencies and *reconstructs* a picture from them. The mathematics that makes this reconstruction possible is called the Fourier transform, and we are going to spend a good deal of time getting comfortable with it.

Now, here is what your project does. You are going to build a desktop application that simulates this entire process on ordinary images, so that you can see every step with your own eyes. First, the application takes an image and converts it into its frequency representation. That frequency representation has a special name, *k-space*, and you will be hearing that name constantly from now on. Next, the application lets the user damage or modify that frequency data in all the ways a real scanner does. A real scanner might skip lines of data in order to scan faster. It might pick up electrical noise. It might sample the data along circles and spirals instead of along straight rows. Your application can do each of these things on purpose. After the damage is done, the application reconstructs the image from the damaged data, and then it *measures* exactly how much image quality was lost along the way. Every one of these effects is controlled by a slider or a dropdown, so the user can move a control and watch the result change instantly.

Analogy

Here is a picture that may help you hold all of this in your head. Think of the scanner as a musician with perfect pitch who is sitting in a concert hall with their eyes closed. This musician cannot *see* the orchestra, and in our story the orchestra plays the role of the image. What the musician can do is hear every individual note being played, and those notes play the role of the frequencies. Now suppose the musician writes down the full list of notes along with how loudly each one was played. With that list in hand, someone could later reconstruct exactly what the orchestra sounded like. In our project, *k-space* is that written list of notes. The Fourier transform is the perfect-pitch skill itself. It is what lets you go from “the sound of the whole orchestra” to “a list of individual notes,” and it also lets you go back the other way, and in neither direction is anything lost.

Now let us imagine a few things going wrong, because this is where the analogy really earns its keep. Suppose the musician gets lazy and only writes down every second note. In our project we call that undersampling. Suppose the hall has an air conditioner humming in the background that gets mixed into what the musician hears. In our project that is noise. Suppose the musician scribbles the notes in a spiral around the page instead of in neat rows. In our project we call that non-Cartesian sampling. Each of these problems corrupts the reconstruction, and the important thing to notice is that each one does so in a *specific, predictable* way. Your application is going to demonstrate every one of them.

1.2 The five ideas the entire application rests on

It may feel like the roadmap is asking you to learn a great many separate things. Every module, every slider, and every formula in it, though, is really an application of just five ideas. I would like you to read these five ideas now, and then come back and read them again after you finish each chapter. Each time you return, you will find that a little more of the list has become obvious to you.

1. **An image is a matrix of numbers.** Take a grayscale image of size 256×256 . That image is nothing more than a table containing 65,536 brightness values, and each of those values sits somewhere between 0.0, which means black, and 1.0, which means white.
2. **Any image can be rewritten as a sum of waves.** The smooth, slow waves are the ones that capture the overall shapes and the contrast of the picture. The fast, tight waves are the ones that capture the edges and the fine texture. When you write down the recipe that says “which waves to use, how strong each one should be, and how far each one is shifted,” that recipe *is* k -space. We will build this up carefully in Chapter 2.
3. **Removing waves from the recipe damages the picture in predictable ways.** Here is what we mean by that. If you remove whole rows from the recipe, ghost copies of the image appear, and that effect is called aliasing. If you chop off the outer part of the recipe abruptly, rippling rings appear near the edges of objects, and that effect is called Gibbs ringing. You will meet both of these in Chapters 3 and 4.
4. **The reconstruction machinery only accepts recipes written on a neat square grid.** This matters because a scanner does not always collect its recipe that way. If the scanner collected the recipe along spokes or spirals, we must first *re-grid* it, which means moving the data back onto a square grid. We have to do this carefully, because spokes naturally crowd together in the middle, and we need to compensate for that crowding. This is the subject of Chapter 5.
5. **Damage can be measured with three numbers.** Those three numbers are MSE, PSNR, and SSIM. What they do for us is turn a vague feeling like “this image looks worse” into quantities that you can actually plot against the position of a slider. You will learn all three in Chapter 6.

1.3 The data-flow pipeline, drawn as a picture

Let us now look at how the pieces connect. Every interaction in the finished application, which in practice means every time the user drags a slider, pushes an image through the pipeline shown below exactly once. I would encourage you to keep this figure in your head as you read, because each chapter of this guide simply zooms into one of these boxes and explains what happens inside it.

1.4 The files you will create, and what each one is responsible for

The application is split into several small files, and each file is responsible for one of the boxes in Figure 1.1. I want to explain why we bother to split things up this way, because it is the thing that makes a six-week project survivable. When each file owns just one job, you can write that file and test it *on its own*, from a terminal, long before any GUI exists. That means you never have to debug the mathematics and the user interface at the same time.

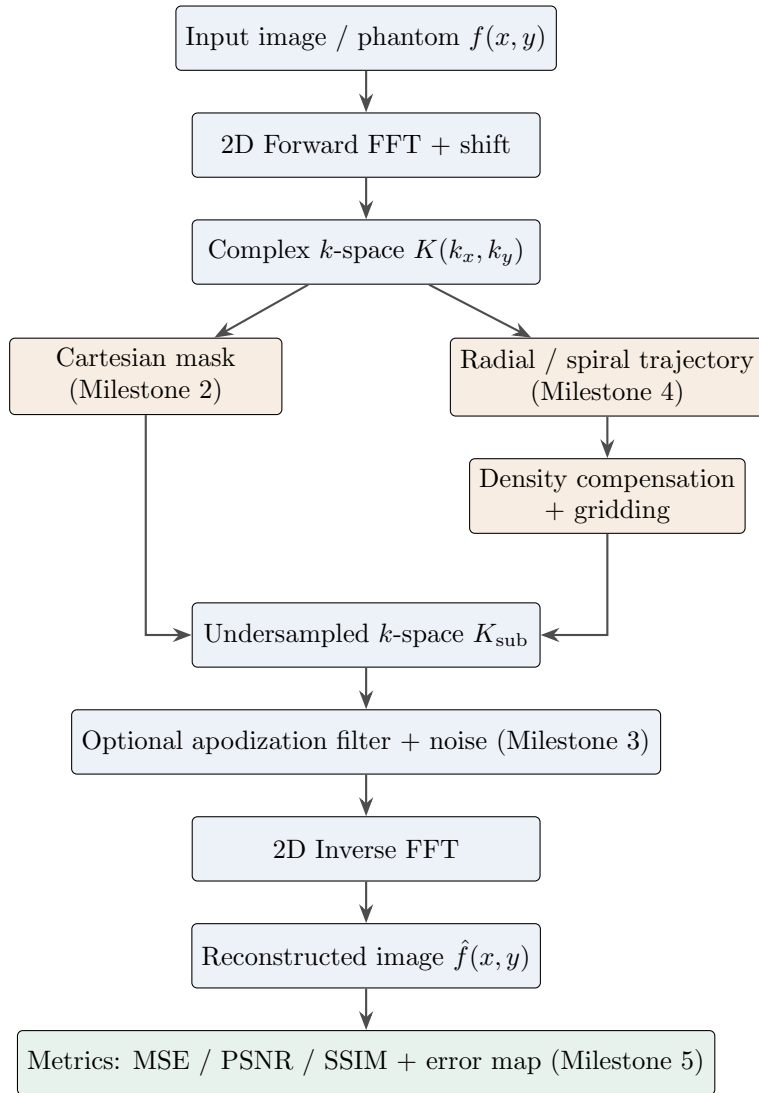


Figure 1.1: The complete data-flow pipeline of the simulator. The GUI that you build in Milestone 6 is simply a set of sliders that re-runs this pipeline, together with four panels that display its intermediate results.

File	Owns	Guide chapter
<code>kspace_core.py</code>	FFT, IFFT, shifting, log display, phantom loading	2
<code>cartesian_sampling.py</code>	Binary masks, acceleration factor, PSF	3
<code>filters.py</code>	Hamming/Gaussian windows, noise injection	4
<code>trajectories.py</code>	Radial and spiral coordinates, density weights	5
<code>gridding.py</code>	Interpolation of scattered points onto the grid	5
<code>metrics.py</code>	MSE, PSNR, SSIM, error heatmaps	6
<code>app.py</code>	Windows, panels, sliders, worker threads	7

Key Takeaways

- The scanner we are simulating records **frequencies**, and it never records pictures. Turning those frequencies back into a picture is a purely mathematical step, and the mathematics that does it is the 2D Fourier transform, which lives in `kspace_core.py`.

- Your whole application is really just one pipeline, the one drawn in Figure 1.1, and the application runs that same pipeline over and over again with different settings each time.
- Every “advanced” term you will come across in the roadmap, such as PSF, apodization, density compensation, and gridding, is a small idea that happens to wear an intimidating name. Each of those ideas gets its own gentle chapter in this guide, so there is no need to worry about them yet.

Milestone 1 — Images, Waves, and the Fourier Transform

This is the foundation chapter of the whole guide. Everything we build in the later milestones rests on the ideas introduced here, so if you understand this chapter well, every later chapter will feel like a variation on something you already know. Please take your time with it. There is no rush, and it is completely normal to read some of these sections twice.

2.1 Step 1: An image is just a matrix

The Concept in Plain English

Let us start with the most basic question of all: what is an image, as far as the computer is concerned? A grayscale digital image is simply a rectangular table of numbers. Each cell of that table is one **pixel**, and the number stored inside the cell tells you how bright that pixel is. In this project we standardize those numbers so that they always lie between 0.0, which means pure black, and 1.0, which means pure white. So a 256×256 image is a matrix with 256 rows and 256 columns, which works out to 65,536 numbers in total. That really is all an image is. The reason this matters so much is that everything we do to images in this project turns out to be *matrix arithmetic*. We will be adding matrices together, multiplying them element by element, and transforming them into other matrices. Once you see the image as a grid of numbers, none of those operations should feel mysterious.

When we write this down mathematically, we call the image $f[m, n]$. Here m is the row index and n is the column index, and for an $N \times N$ image each of them runs from 0 up to $N - 1$. Before we go any further, here are two practical notes that will save you a good deal of debugging later on.

- **Color images must be collapsed to one channel.** A color pixel is not one number but three numbers, one each for red, green, and blue. MRI images, on the other hand, are single-channel. So whenever a color image comes in, we convert it to a single brightness value using the standard luminance formula. This formula weights the three channels in the same way the human eye does, which is why the green channel gets the largest share:

$$Y = 0.299 R + 0.587 G + 0.114 B. \quad (2.1)$$

- **Sizes should be powers of two.** The Fast Fourier Transform, which we meet properly in Step 3, runs at its very fastest when the matrix dimensions are of the form 2^k , for example 256 or 512. For that reason we resize every input image to 256×256 , or pad it with zeros until it reaches that size.

2.2 Step 2: What is a “spatial frequency”?

The Concept in Plain English

You already know what frequency means from sound. A high-pitched note is a pressure wave that wiggles quickly in *time*, and a low-pitched note is one that wiggles slowly. A **spatial** frequency is exactly the same idea, except that we apply it to *space* instead of time. To picture it, imagine painting smooth stripes across a canvas, where the paint fades gently from black to white and back to black again, like a wave. If the stripes are wide and lazy, we say the pattern has a *low* spatial frequency. If the stripes are thin and packed tightly together, we say it has a *high* spatial frequency. A striped pattern of this kind is called a **grating**, and you can think of it as the two-dimensional equivalent of a single pure musical note.

A grating is completely described by three properties. The first is its **frequency**, which tells you how many stripe cycles fit across the image. The second is its **orientation**, because the stripes can run vertically, horizontally, or at any angle in between. The third is its **phase**, which tells you where the bright stripe sits. Changing the phase slides the whole pattern sideways without changing anything else about it.

Analogy

Here is the central claim of Fourier analysis, and it is worth reading slowly. *Any* image at all, whether it is a brain scan, a photograph of your cat, or anything else you can think of, can be built by stacking up enough gratings of different frequencies, orientations, phases, and strengths. This is the same idea as saying that any chord you hear can be built by playing the right single notes at the same time. Now think about what the Fourier transform does in this picture. It is the machine that takes the finished image and hands you back the complete recipe, meaning the exact list of gratings that went into it and how strong each one is. When we talk about *k*-space later in this chapter, we are talking about precisely this recipe.

To make this concrete, let us look at the idea in one dimension first, where we can draw it. In the figure below, watch how adding just a few pure sine waves on top of each other already starts to build a sharp-edged square wave. The lesson to take away is that edges, and sharp features in general, require *many high frequencies* before they can appear.

Why This Matters

This one figure quietly explains two phenomena that you will meet later in the project, so let us point them out now, before you even get there.

- Suppose we *delete the high frequencies*, which is something we will do on purpose in Chapters 3 and 4. Look at the figure and ask what happens to the edges. Without the high-frequency terms, the edges simply cannot be formed sharply anymore, and so the image blurs.
- Now suppose that instead of fading the series out gently, we *cut it off abruptly* at some term. When we do that, the sum overshoots and ripples near every edge. You can already see small wiggles in the green curve in the figure. That rippling has a name. It is called **Gibbs ringing**, and we will deal with it directly in Chapter 4.

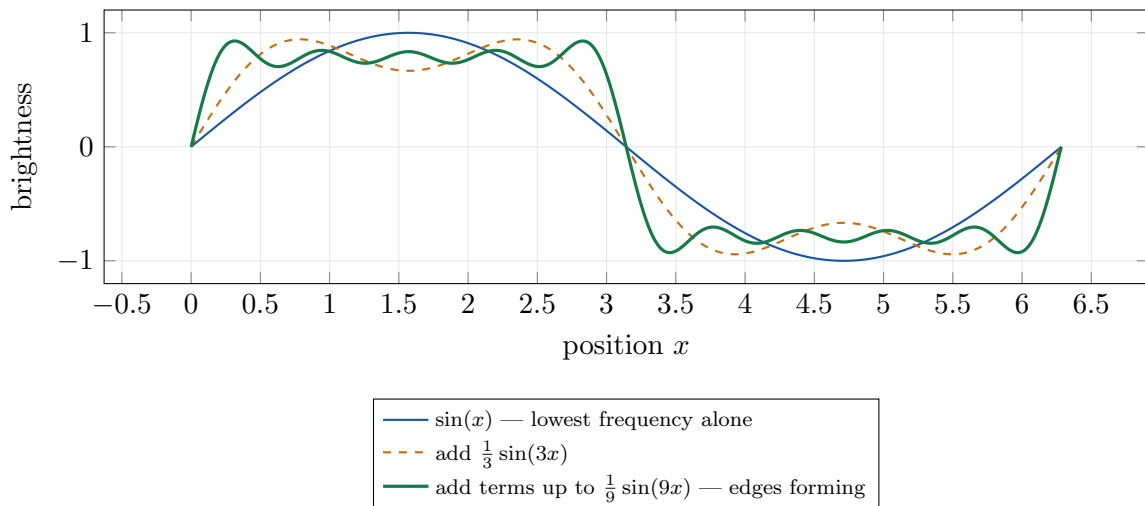


Figure 2.1: Summing sine waves builds sharper and sharper shapes. The low frequencies sketch the rough outline, and the high frequencies carve out the sharp edges. Exactly the same thing is true in two dimensions for images.

2.3 Step 3: The Fourier transform, gently

2.3.1 The continuous version (the “textbook” formula)

Let us begin with the version you would find in a textbook. For a continuous image $f(x, y)$, the two-dimensional Fourier transform and its inverse are written as follows:

$$K(k_x, k_y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-i2\pi(k_x x + k_y y)} dx dy, \quad (2.2)$$

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} K(k_x, k_y) e^{+i2\pi(k_x x + k_y y)} dk_x dk_y. \quad (2.3)$$

Please do not let the integrals intimidate you. The best way to read Equation (2.2) is as a procedure for extracting the recipe we talked about in Step 2. Let us walk through it one piece at a time.

- First, look at the term $e^{-i2\pi(k_x x + k_y y)}$. This term **is** a grating. It is a single wavy pattern that completes k_x cycles per unit distance in the horizontal direction and k_y cycles per unit distance in the vertical direction. If the complex exponential looks unfamiliar, remember Euler’s formula, $e^{i\theta} = \cos \theta + i \sin \theta$. What it tells us is that a complex exponential is nothing more than a cosine and a sine travelling together as a pair.
- Next, notice that we multiply the image by that grating and then integrate, which means we sum the products over the whole image. This is really asking one single question: “*How much of this exact grating is present in this image?*” If the image contains a lot of that pattern, the products reinforce one another and the integral comes out large. If the image does not contain that pattern, the positive and negative contributions cancel each other out, and the integral comes out close to zero.
- Finally, think about what $K(k_x, k_y)$ is. At each position (k_x, k_y) it stores the answer to that question for the grating with that particular frequency and orientation. So the whole function K is the complete recipe for the image. This function is what we call **k -space**, and it is the object that the rest of this project revolves around.

Equation (2.3) does the reverse job. You take every grating, scale it by its entry in the recipe, add all of them together, and the original image reappears. If you compare the two formulas

carefully, you will notice that the only difference between them is the sign in the exponent. A minus sign means we are taking the image apart into gratings, and a plus sign means we are putting the gratings back together into an image.

2.3.2 The discrete version (the one your computer runs)

A computer cannot hold a continuous function. What it can hold is a matrix, so in practice we use the Discrete Fourier Transform, usually shortened to DFT. For an $N \times M$ image it looks like this:

$$K[u, v] = \sum_{m=0}^{N-1} \sum_{n=0}^{M-1} f[m, n] e^{-i2\pi(\frac{um}{N} + \frac{vn}{M})}, \quad (2.4)$$

$$f[m, n] = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} K[u, v] e^{+i2\pi(\frac{um}{N} + \frac{vn}{M})}. \quad (2.5)$$

Notice that the only thing that changed is that the integrals became sums. Everything else is identical to the continuous version. The indices u and v now count *how many full cycles* the grating completes across the image. So $u = 0$ means there is no variation at all, which is a perfectly flat sheet. Then $u = 1$ means one full wave across the image, $u = 10$ means ten tight waves across the image, and so on upward.

You will hear the name **FFT**, which stands for Fast Fourier Transform, all the time. It is important to understand that the FFT is not a different transform. It is a clever algorithm that computes exactly Equation (2.4), but it does so in $\mathcal{O}(N^2 \log N)$ operations rather than the $\mathcal{O}(N^4)$ operations that the naive approach would need. That enormous speed difference is what makes real-time sliders possible in our GUI later on, and it is also the reason we prefer power-of-two image sizes, as mentioned back in Step 1.

2.3.3 k -space values are complex numbers, and that matters

Each entry $K[u, v]$ of k -space is a **complex number**, which we can write as $a + bi$. You might wonder why a complex number is needed here. The reason is that each grating needs *two* facts recorded about it, and a complex number is a convenient container that stores exactly two numbers. Those two facts are the magnitude and the phase:

$$\textbf{Magnitude} \quad |K[u, v]| = \sqrt{a^2 + b^2} \quad \text{— } \textit{how strong is this grating?} \quad (2.6)$$

$$\textbf{Phase} \quad \angle K[u, v] = \text{atan2}(b, a) \quad \text{— } \textit{how shifted is it sideways?} \quad (2.7)$$

Why This Matters

There is a classic experiment here that is well worth reproducing in your own app once it is working. First, reconstruct an image using only the magnitudes, which you do by setting every phase to zero. What you get is an unrecognizable blob. Then do the opposite and reconstruct using only the phases, which you do by setting every magnitude to one. This time, the edges and outlines of the original image are clearly visible. The lesson is surprising but important: **phase carries the structure, and magnitude carries the contrast strength**. This is exactly why we must keep k -space complex all the way through the pipeline, and only take magnitudes at the very final step when we display something on the screen.

2.4 Step 4: fftshift, or putting the recipe's center in the middle

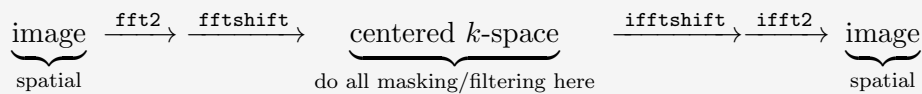
Here is a detail that trips up almost everyone the first time. The raw output of the FFT stores the zero-frequency entry in the *top-left corner* of the matrix. That zero-frequency entry is called

the **DC component**, and you can think of it as the flat “average brightness” grating. The negative frequencies, meanwhile, are wrapped around to the far side of the matrix. There is nothing deep about this arrangement. It is just a storage convention. But for humans looking at the picture, and also for the masks we will write later that assume the center of the matrix is the low-frequency region, we want the DC term sitting in the *middle* of the matrix, with the frequency increasing outward in every direction.

The function `fftshift` performs exactly this rearrangement. It swaps the quadrants of the matrix so that the DC entry lands at the center, and the function `ifftshift` undoes that swap. This brings us to what is probably the single most important rule of the whole project.

Key Takeaways

The sandwich rule. Every trip into k -space and back out again follows the same symmetric pattern:



If you ever see a reconstruction that looks as though a checkerboard was multiplied into your image, the explanation is almost certainly that you forgot one of the two shifts. Keep this in mind, because it is the single most common bug in the entire project.

2.5 Step 5: Why we display $\log(1 + |K|)$

When you first try to look at k -space, you will run into a practical problem. The DC component and its low-frequency neighbors are *astronomically* larger than everything else, often a million times larger. The reason is that most images are dominated by smooth regions, and smooth regions are built almost entirely out of low frequencies. So if you display $|K|$ directly, all you will see is one white pixel in the center and blackness everywhere else. The fix is to apply logarithmic compression before displaying:

$$V[u, v] = \log(1 + |K[u, v]|). \quad (2.8)$$

Let us look at the two pieces of this formula. The “1+” guarantees that the argument of the logarithm is never zero, which matters because $\log 0$ would be $-\infty$. The logarithm itself then squeezes the enormous dynamic range down into something a screen can actually show. One warning, though. This is for *display only*. You should never feed the log version back into the reconstruction, because it is no longer the real recipe.

2.6 Step 6: The anatomy of k -space

Once you know this geography, you can *predict* what any manipulation of k -space will do to the image before you even run it. If you delete the periphery, you will get an image that is blurry but still recognizable, because you have kept the contrast and thrown away the detail. If you delete the center instead, you will get a ghostly outline showing only the edges, because now you have kept the detail and thrown away the contrast. More generally, the shape of a mask in k -space determines the shape of the artifact you see in the image. We will use this idea again and again in the later milestones.

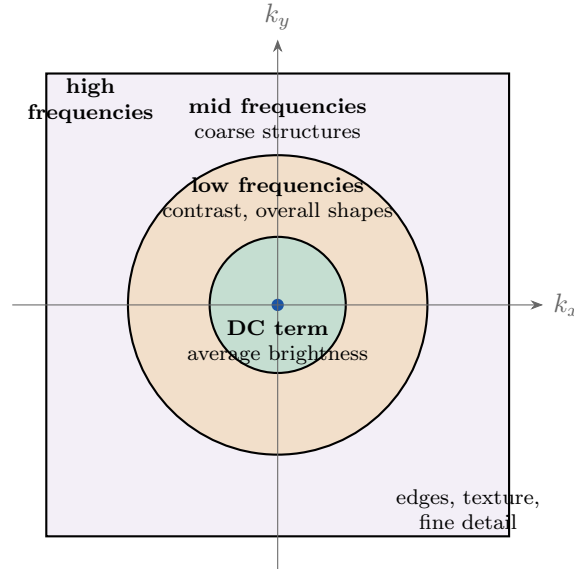


Figure 2.2: The anatomy of centered k -space. Distance from the center equals spatial frequency. The tiny central region carries almost all of the image *energy*, which we see as contrast, while the vast periphery carries the *detail*.

2.7 Step 7: Parseval’s theorem, your built-in correctness test

Parseval’s theorem says that the total “energy” of an image is preserved by the Fourier transform, up to a known scaling constant. Written out, it looks like this:

$$\sum_{m,n} |f[m,n]|^2 = \frac{1}{NM} \sum_{u,v} |K[u,v]|^2. \quad (2.9)$$

In plain English, rewriting the image as a recipe of gratings neither creates anything nor destroys anything. It is a lossless change of viewpoint. One way to picture it is weighing your ingredients before mixing them and then weighing the finished mixture afterwards, and finding that the two weights agree. The practical payoff is that Equation (2.9) hands you a *free automated test*. You compute both sides, and if they differ by more than floating-point dust, which is roughly 10^{-10} in relative terms, then you know there is a bug somewhere in your FFT plumbing.

2.8 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

All of the code for this milestone lives in one module, `kspace_core.py`. Work in a plain script or a Jupyter notebook with `matplotlib` for now. There should be absolutely no GUI code at this stage, because we want to get the mathematics right before we worry about buttons and sliders.

A. The image loader

- Start by writing a loader that accepts a file path and returns a clean matrix. Inside it, open the image, convert it to grayscale using the luminance formula from Step 1, and convert the data type to 64-bit floating point.
- Resize the result to 256×256 . You will need to decide on a policy for inputs that are not square, and you should write that policy down in a comment. The simplest option is to resize both dimensions directly. The better option is to pad the image

with zeros to make it square first, so that the shapes in it are not distorted.

- Normalize the matrix by dividing by its maximum value, so that every pixel lies in $[0, 1]$, just as we agreed in Step 1.
- Add a second loader that produces the **Shepp–Logan phantom**. This is a standard synthetic “fake head” image that is available in `scikit-image`, and having it means you always have a canonical test image whose properties you know in advance.

B. The forward transform

- Write a function that takes the image matrix and returns centered k -space. To do that, apply the 2D FFT, and then apply the quadrant-swapping shift from Step 4 so that the DC term sits at the center. Keep the result *complex*. Remember from Step 3 that the phase carries the structure, so you must not take magnitudes here.
- On top of that function, write two small helpers. The first one returns the **magnitude spectrum**, with an optional flag that applies the log compression from Step 5 for display. The second one returns the **phase spectrum**, which you compute with the two-argument arctangent of the imaginary part and the real part.

C. The inverse transform

- Write a function that takes any centered k -space matrix and returns the reconstructed image. Following the sandwich rule, you undo the shift first, then apply the 2D inverse FFT, and then take the magnitude, meaning the absolute value, of the complex result.
- Make sure this function accepts *any* k -space matrix as its input. The reason is that the later milestones will feed it masked, filtered, noisy, and re-gridded versions of k -space, and you want one function that handles all of them.

D. The frequency-probing experiment (builds intuition you will need)

- Write a function that keeps only a central disc of k -space, with a radius of r pixels, sets everything outside that disc to zero, and then reconstructs the image. What you have just written is a **low-pass filter**.
- Now write its twin. This one deletes the central disc and keeps the periphery instead. That is a **high-pass filter**.
- Reconstruct the phantom with each of these, at several different radii, and save the side-by-side plots. It is important that you observe the result with your own eyes. The low-pass version should come out blurry but recognizable, and the high-pass version should come out as a dark image that shows only the edges and outlines. This is exactly what the anatomy in Step 6 predicted.

How to Test It Before Moving On

- **Round-trip test:** take an image, run it through the forward transform and then straight through the inverse transform with no modification in between. The maximum absolute pixel difference from the original must be below 10^{-12} . If it is not, the first place to look is your shift sandwich.
- **Parseval test:** verify that Equation (2.9) holds to within 10^{-10} relative error, just as we discussed in Step 7.
- **Visual test:** display the log-magnitude spectrum of the phantom. You should see a bright star-like glow exactly at the center of the matrix, fading outward, with faint spokes running through it. If the glow is at the four corners instead, that tells you

that you forgot the centering shift.

- **DC sanity test:** the magnitude at the exact center pixel should equal the sum of all the image pixels. The reason is that this is literally what the $u=v=0$ term of Equation (2.4) computes, since the grating there is flat. Check it numerically.

Self-Check: Can You Answer These?

1. In one sentence: what does the value stored at position (u, v) of k -space tell you?
2. Why must k -space entries be complex numbers? What do the magnitude and the phase each encode?
3. What exactly does `fftshift` move, and why do we want it moved?
4. Why is the center of k -space so much brighter than the rest, and how do we manage to display it anyway?
5. If you delete the outer ring of k -space, what happens to the image, and *why*, in terms of Figure 2.1?
6. State Parseval's theorem in plain English. How would you use it as a unit test?
7. Which reconstruction looks more like the original image: the magnitude-only one or the phase-only one? What does that tell you about where structure lives?

Milestone 2 — Sampling, Undersampling, and Aliasing

3.1 Why scanning takes time, and why we are going to cheat

The Concept in Plain English

Let us start with something about real scanners that is easy to miss if you only ever work with saved data files. A real MRI scanner cannot capture all of k -space in one go. Instead, it acquires the data **one horizontal line at a time**. The machine plays a magnetic gradient, records a single row of frequency samples, then waits for the physics inside the patient to settle back down, and only then records the next row. Each of these rows has a name. It is called a **phase-encoding line**. If you need all 256 lines of a full acquisition, the whole process can take minutes. During those minutes the patient is moving a little, breathing, and generally getting uncomfortable. Because of this, there is enormous clinical pressure to *skip some of the lines* so that the scan finishes faster.

When we skip lines on purpose, we call it **undersampling**. Suppose we acquire only every second line. Then we have done half the work, so we scan twice as fast. The amount of speed-up we gain has its own name, the **acceleration factor**, and it is defined like this:

$$R = \frac{\text{lines a full scan would need}}{\text{lines actually acquired}} = \frac{N_{\text{total}}}{N_{\text{sampled}}}.$$

Now, here is the catch. Mathematics does not let us throw away information for free. The price we pay for the skipped lines is a strange and rather beautiful artifact that is called **aliasing**. The whole point of this milestone is to understand exactly why aliasing happens, and also why it looks the particular way it looks when it shows up in an image.

3.2 The Nyquist rule in plain English

Analogy

You have almost certainly seen aliasing before, even if you did not know the name for it. Think about an old western movie. Sometimes the wheels of a wagon appear to spin *backwards*, even though the wagon is clearly rolling forwards. Here is what is going on. The camera takes a picture of the world 24 times every second. If the spokes of the wheel are turning faster than the camera can keep up with, then each snapshot catches the spokes in a position that does not line up with the previous one in the way your brain expects. Your brain is fed a series of misleading snapshots, and it perceives a slower motion, or even a reversed one. So a fast thing, when it is sampled too rarely, ends up *masquerading as a slow thing*. That is exactly what the word “alias” means. It literally means “false identity.” Keep this wagon wheel in mind, because the same thing is about to happen to our k -space data.

The precise version of this idea is called the **Nyquist–Shannon criterion**. It says that if you want to capture a signal faithfully, you must take samples at least twice as often as the

highest frequency that is present in the signal. In MRI, the “samples” we are talking about are the k -space lines from the scanner, and so the rule takes on a spatial form. The spacing between the acquired k -space lines, which we write as Δk , determines the size of the region that the image can faithfully represent. That region is called the **field of view**, and the relationship is simply

$$\text{FOV} = \frac{1}{\Delta k}. \quad (3.1)$$

It helps to read this equation as a see-saw. When the k -space lines are *finely spaced*, the field of view is large, and there is plenty of room for the whole head to fit inside it. When the lines are *coarsely spaced*, which is exactly what happens when we skip some of them, the field of view *shrinks*. If we skip lines by a factor of R , the new, effective field of view is

$$\text{FOV}_{\text{eff}} = \frac{\text{FOV}}{R}. \quad (3.2)$$

And now we arrive at the punchline. The object we are imaging does *not* shrink along with the field of view. The head is still the same size it always was. So we now have an object that is larger than the region the mathematics can represent, and that object has nowhere to go. What the mathematics does is wrap it around, in much the same way that a word processor wraps a line of text that is too long to fit on the page. Copies of the anatomy fold back on top of the original. Radiologists have two names for this effect. They call it **wrap-around**, or **ghosting**.

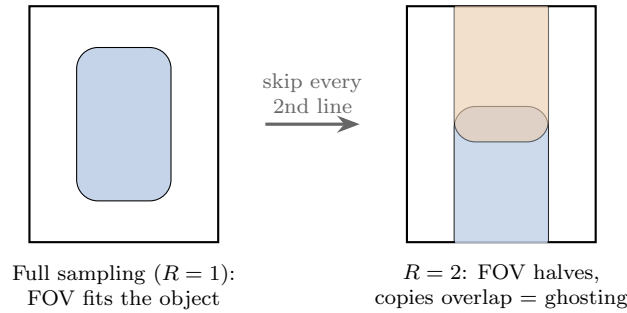


Figure 3.1: Uniform undersampling by $R=2$ halves the effective field of view along the skipped direction. The anatomy no longer fits, so a shifted duplicate wraps around and overlaps the original.

3.3 Sampling masks: undersampling is just a multiplication

Here is a practical question. In our project we already have the complete k -space sitting in memory, because we computed it ourselves in Milestone 1. So how do we *simulate* a scanner that skipped some of the lines? The answer is a **binary mask**. A mask is a matrix, which we will call $S[u, v]$, that has exactly the same size as the k -space. Wherever a sample was “acquired” the mask holds a 1, and wherever a sample was “skipped” the mask holds a 0. Once you have such a mask, undersampling becomes a single element-wise multiplication:

$$K_{\text{sub}}[u, v] = K[u, v] \cdot S[u, v]. \quad (3.3)$$

Take a moment to appreciate how simple this is. This one multiplication is the entire trick of Milestone 2. Everything else we do in this chapter is really just choosing interesting patterns of ones and zeros to put into the mask. Your app is going to implement four such patterns, and you can see all four of them side by side in the figure below.

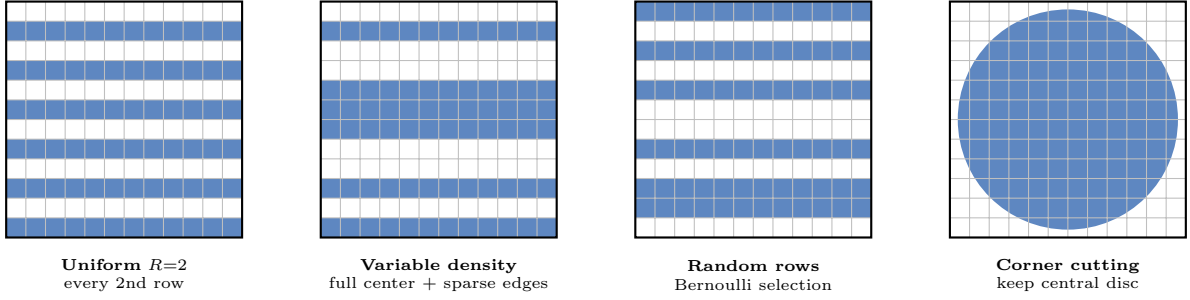


Figure 3.2: The four Cartesian sampling masks (blue = acquired = 1, white = skipped = 0). Each produces a visually distinct artifact in the reconstruction.

1. **Uniform subsampling.** This is the simplest pattern of all. We keep every R -th row and throw away the rest. Written as a formula, the mask is

$$S_{\text{uniform}}[u, v] = \begin{cases} 1 & \text{if } v \bmod R = 0 \\ 0 & \text{otherwise.} \end{cases} \quad (3.4)$$

This is the pattern that produces the classic, *coherent* ghost copies you saw in Figure 3.1. The reason is worth saying out loud. We skipped rows in a perfectly regular rhythm, and a regular pattern of skipping always produces a regular pattern of ghosts.

2. **Variable-density sampling (VDS).** In this pattern we are a little smarter about which rows we keep. We fully sample a central band of rows. Those central rows are the low frequencies, and if you look back at Figure 2.2 you will remember that nearly all of the energy in k -space lives there. Then we sample the outer rows only sparsely, keeping each of them with some probability. Because the important central rows are all present, this pattern preserves the overall contrast of the image far better than uniform sampling does at the same acceleration factor R . The artifact is still there, but it becomes milder and less structured.
3. **Random subsampling.** Here we decide, for each row independently, whether to keep it by flipping a coin that comes up “keep” with probability $1/R$. Here is the intuition for what this does to the artifact. The ghost copies in the uniform case were crisp precisely because we skipped rows in a perfectly regular rhythm, and a regular rhythm in k -space always turns into a clean, repeating pattern in the image. The moment we choose the rows randomly, there is no rhythm left for the mathematics to turn into clean copies, so the same lost information shows up instead as a scattered, noise-like grain spread across the whole picture. You may be interested to know that this idea goes a long way. Random undersampling turns aliasing into something that looks like “noise,” and clever algorithms are good at removing noise. That observation is the foundation of a whole field called *compressed sensing* MRI.
4. **Corner cutting, also called a circular mask.** In this pattern we keep only the entries that satisfy $u^2 + v^2 \leq r_{\text{max}}^2$, where the distance is measured from the center of k -space. Notice that this is a different kind of pattern from the other three. We are not skipping rows here. Instead we are truncating frequencies, which makes this mask a pure low-pass filter. It blurs the image, because the fine detail lives in the high frequencies we removed. It also does something else. Because the cutoff is abrupt, going from fully kept to fully removed in a single step, it introduces the Gibbs ringing that we are going to fix in Milestone 3.

3.4 The Point Spread Function: predicting the artifact before you see it

The Concept in Plain English

We now come to what I think is the most elegant idea in this whole milestone. Up to now we have been asking the question, “what does this mask do to *my image*?” Let us instead ask a purer and simpler question. “What does this mask do to a *single bright dot*?” The answer to that question has a name. It is called the **Point Spread Function**, or **PSF** for short. Think about what a perfect imaging system would do with a dot. It would leave the dot exactly as a dot. A mask does something different. It smears the dot out into some pattern. And here is the important part. *That same pattern gets stamped onto every single point of any image* you reconstruct through the mask. So if you know what happens to one dot, you know what happens to everything.

The mathematics behind this is a result called the **convolution theorem**, and it is the most important theorem of the entire course. In words, it says that *multiplication in k -space is the same thing as convolution in image space*. Written out properly, it looks like this:

$$K_{\text{sub}} = K \cdot S \iff \hat{f} = f ** \text{PSF}, \quad \text{PSF} = \mathcal{F}_2^{-1}\{S\}, \quad (3.5)$$

Let us go through the symbols. The double star `**` stands for 2D convolution. If you have not met convolution before, here is the recipe it describes. You take a copy of the PSF and stamp it down at every pixel of the image f . Each stamped copy is scaled by the brightness of the pixel it sits on. Then you add up all of those stamped copies, and the result is the reconstructed image $\hat{f}$. The last part of the equation tells you where the PSF comes from. You simply take the inverse 2D Fourier transform of the mask S , treating the mask as if it were a k -space matrix of its own.

Once you understand this, artifact analysis turns into a lookup table. You look at the PSF of a mask, and it tells you what artifact to expect:

Mask	PSF looks like	So the artifact is
Uniform R	one central spike + $R-1$ side spikes	$R-1$ shifted ghost copies
Random rows	central spike + scattered low ripple	noise-like grain
Corner cut	a blob with concentric rings (a jinc)	blur + ringing at edges
Full sampling	a single perfect spike	no artifact at all

Let us look closely at the first row of that table, because it closes a loop we opened at the start of the chapter. The side spikes in the PSF of the uniform mask *are* the ghosts. Think about what it means to convolve the image with a PSF that is “a spike here, plus a spike shifted by FOV/R .” Stamping that pattern at every pixel gives you the original image, plus a copy of the image shifted by FOV/R . That is precisely the wrap-around we drew in Figure 3.1. So the convolution theorem does not merely describe aliasing after the fact. It *explains* why aliasing has to look the way it does.

3.5 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in two files. The mask generation goes in `cartesian_sampling.py`, and the PSF study goes in `psf_analyzer.py`. There is still no GUI at this stage, so you will be working with saved plots, just as you did in Milestone 1.

A. The mask generator

- Start by creating a generator class or module. Every function in it should return a matrix of zeros and ones that has the same shape as your k -space, which in our case is 256×256 .
- **Uniform:** begin with a matrix of all zeros. Then walk through the rows and set an entire row to one wherever the row index is divisible by R . The value of R should be a parameter that the caller passes in.
- **Variable density:** first set a central horizontal band of rows fully to one. The height of that band should be a parameter, and something like the middle 16–32 rows is a sensible choice. Then, for every row outside the band, draw a random number and keep the row if that number falls below a keep-probability. You derive that keep-probability from the R the user requested.
- **Random:** keep each row independently with probability $1/R$. Also accept an optional random seed as a parameter. The reason is that you will want to repeat experiments later, and a seed makes the random choices reproducible.
- **Corner cutting:** build two coordinate matrices. One holds each entry's horizontal distance from the center of the matrix, and the other holds the vertical distance. Then set to one every entry whose radius $\sqrt{u^2 + v^2}$ is below the cutoff parameter.
- Finally, add a small reporting helper. It should take any mask and return its *actual* acceleration factor, which you compute as the total number of entries divided by the number of ones. This matters because for the VDS and random masks the R you actually achieve is different from the R that was requested, so you should display both numbers.

B. Wiring masks into the pipeline

- Take the centered k -space that you produced in Milestone 1, multiply it element-wise with the mask, and feed the product into your existing reconstruction function. Notice that nothing inside `kpace_core.py` should need to change for this to work. If you find that it does need to change, that is a sign that the interfaces you wrote in Milestone 1 were too narrow, and it is worth going back to widen them.

C. The PSF analyzer

- Write a function that takes a mask and treats it as if it were a k -space matrix. Push it through your inverse transform, using the proper shift sandwich from Milestone 1, and the result is the PSF image.
- When you display the magnitude of the PSF, use a log scale. The reason is that the side lobes are tiny compared to the main spike, and if you plot on a linear scale you will see nothing but the one spike.
- Also extract the central row of the PSF and plot it as a simple 1D curve. On this curve the main spike and the side spikes are unmistakable.
- For the uniform mask, check that the geometry matches the prediction. At acceleration R you should count exactly R spikes, which is the main spike plus the ghosts, and they should be spaced N/R pixels apart.

D. The comparison experiment (your Week-2 mini-report)

- Fix $R = 4$. Reconstruct the phantom through the uniform, VDS, and random masks. Save a single figure with three columns, showing the mask, the PSF, and the reconstruction for each case.

- Then write three sentences summarizing what you see. You should be able to report coherent ghosts for the uniform mask, noise-like grain for the random mask, and the best contrast preservation for the VDS mask.

How to Test It Before Moving On

- **Identity test:** a mask of all ones must reproduce the original image to within 10^{-12} . This is your undersampling machinery running with no undersampling at all, so nothing should change.
- **Ghost-count test:** with a uniform mask at $R = 2$, the reconstruction must show exactly one extra ghost copy, and it must be shifted by half the image height. With $R = 4$ you should see three ghosts, spaced at quarter-image intervals.
- **Direction test:** if your mask skips *rows*, then the ghosts must appear along the *vertical* axis. If you see them appearing horizontally instead, your mask is transposed.
- **Energy test:** the masked k -space must have strictly less total energy than the original. The reason is simple. You deleted entries, and Parseval's theorem tells us that the image energy must drop accordingly.
- **PSF test:** the PSF of a full mask must be a single spike at the center, and its total energy must not be spread anywhere else.

Self-Check: Can You Answer These?

1. Define the acceleration factor R and explain what a scanner gains and what it loses when you raise it.
2. State the Nyquist criterion as it applies to k -space line spacing and the field of view (Equation 3.1).
3. Why does skipping every second k -space row cause a *half-image* shift of the ghost, rather than some other shift?
4. Why do random masks produce noise-like artifacts while uniform masks produce crisp ghosts? (Hint: what does regularity in k -space imply when you pass it through the convolution theorem?)
5. What is a Point Spread Function, and why does analyzing it let you predict artifacts *without* reconstructing any actual image?
6. Write down the convolution theorem and explain each symbol in Equation (3.5).
7. Why does variable-density sampling preserve image contrast better than uniform sampling at the same R ? (Hint: look again at Figure 2.2 and ask yourself where the energy lives.)

Milestone 3 — Gibbs Ringing, Apodization, and Noise

4.1 The problem: sharp scissors leave ripples

The Concept in Plain English

Let us start with a fact about every real scan that is easy to forget. Every real scan is *finite*. The scanner we are talking about keeps measuring k -space outward from the center, and then at some radius it simply stops, because the scan has to end at some point. From the point of view of the mathematics, that is a rather brutal thing to do. Think back to Figure 2.1, where we saw that a perfectly sharp edge needs infinitely many high-frequency gratings to build it. When the scan stops at some radius, we are taking that infinite recipe of gratings and chopping it off in the middle of a sentence. The image reacts to this abrupt cut by *ringing*. What that means in practice is that every sharp edge in the picture grows a series of faint parallel ripples running alongside it, very much like the rings that spread out around a stone dropped into still water. This effect has a name, **Gibbs ringing**, and you will also hear it called truncation artifact. It is not just a curiosity, either. In a clinical image, a ripple sitting next to the spinal cord can look exactly like a lesion that is not really there, so radiologists care about it a great deal.

Analogy

Here is a way to feel what is going on. Imagine you are playing a song, and you end it by yanking the speaker cable straight out of the wall. You will hear an ugly “pop.” Now imagine you end the very same song with a smooth fade-out instead. There is no pop at all. Notice that the musical content is exactly the same in both cases. The only difference is *how abruptly you stop*. This is exactly the situation in k -space, and apodization is the fade-out. Without it, the frequencies are fully present up to the edge of the measured region and then suddenly absent, which is a cliff. With apodization, we let the frequencies taper gently down to zero as we move outward, which turns the cliff into a slope.

The clearest way to see this phenomenon is to look at it in one dimension first. Suppose we try to reconstruct a perfect step edge using only a finite number of frequencies. What we get is an overshoot right at the edge, followed by a train of ripples. Here is the surprising part. Those ripples *never disappear*, no matter how many more terms you add to the sum. Adding terms only pushes the ripples closer and closer to the edge, while their height stays the same. The figure below shows this directly:

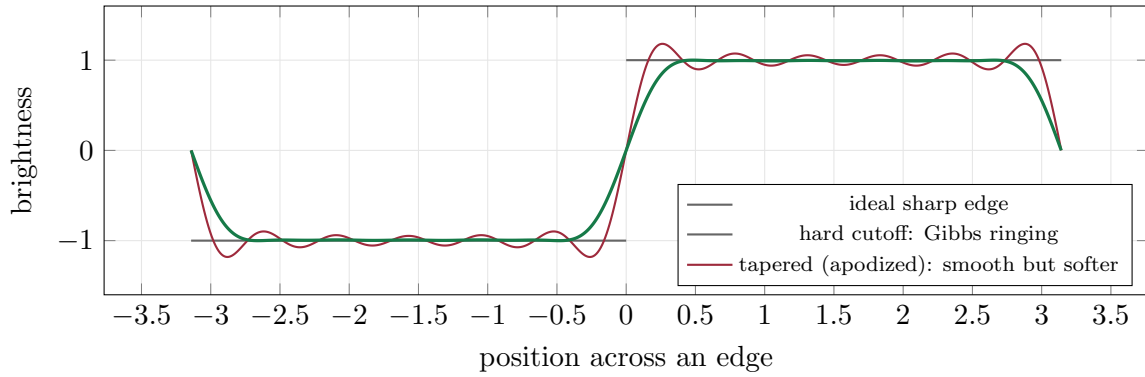


Figure 4.1: Reconstructing a sharp edge from finitely many frequencies. The red curve uses a hard cutoff, and you can see it overshoot and ripple. That is Gibbs ringing. The green curve uses exactly the same frequencies, but tapers them smoothly to zero, and the ripples are gone. The price we pay is that the edge comes out slightly softer. This trade-off is the entire story of apodization.

4.2 The fix: apodization windows

The Concept in Plain English

Now let us look at the fix. An **apodization filter** is a smooth weighting matrix, which we will call $W[u, v]$. The word comes from Greek and literally means “removing the foot,” which is a nice way to remember that it trims the sharp edge off the data. The matrix has the value 1.0 at the center of k -space, and as you move outward toward the periphery it glides smoothly down toward 0.0. We multiply this matrix element by element into k -space before we run the reconstruction. Notice what this does to the high frequencies. They are not amputated any more. Instead, they are *faded* out gradually. The result is that the ringing is suppressed. Of course, there is a price, and the price is a mild blurring of the image. There is no free lunch here, and that is actually the point of this milestone. The job of the app is to let the user *feel* this trade-off for themselves, by moving a slider and watching the image change.

For this project, two classic window shapes are all you need. Let us look at each one in turn:

- **2D Hamming window.** We build this one as the product of two one-dimensional raised cosines, one running across each axis. For a matrix with centered indices u, v , the formula is:

$$W_{\text{Ham}}[u, v] = \left(0.54 + 0.46 \cos \frac{\pi u}{N/2}\right) \left(0.54 + 0.46 \cos \frac{\pi v}{N/2}\right). \quad (4.1)$$

There is one detail worth noticing here. This window does not descend all the way to zero at the edge. Instead it floors at 0.08. That is not an accident or a rounding error. It is a deliberate design choice that balances how much ripple suppression you get against how much blur you introduce.

- **2D Gaussian window.** This is the smoothest taper that mathematics has to offer. It is radially symmetric, which means its value depends only on how far you are from the center, and it comes with one width knob that we call σ :

$$W_{\text{Gauss}}[u, v] = \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right). \quad (4.2)$$

So what does σ actually control? When you make σ small, the Gaussian falls off quickly, which means you are fading out most of the high frequencies, and the reconstructed image

comes out noticeably smoother and blurrier. When you make σ large, the curve stays close to 1.0 almost everywhere, and the window barely touches the data at all. Because this one number has such a visible effect, we are going to hand it to the user as a slider in the GUI.

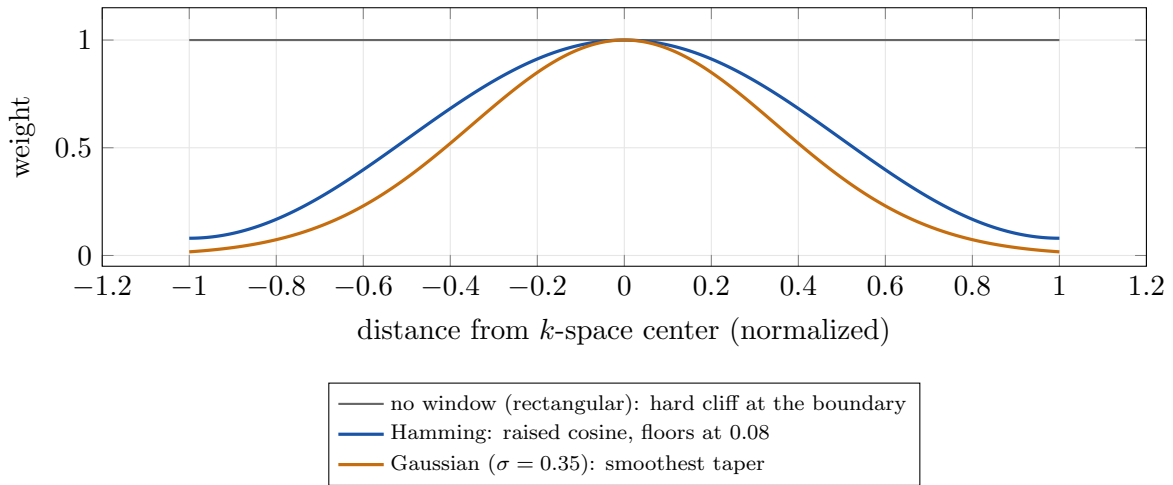


Figure 4.2: Cross-sections of the window options. The rectangular “window” is what you get when you do nothing at all, and it ends in a cliff at the boundary. That cliff is exactly what produces the ringing. Both of the smooth windows trade a controlled amount of blur for the suppression of that ringing.

Why This Matters

You might be wondering why multiplying k -space by a window should have anything to do with blurring the image. The reason is the convolution theorem, which we met in Equation 3.5. That theorem tells us that multiplying two things together in k -space is the same as convolving them in image space. So let us ask what the inverse transform of a Gaussian window looks like. The answer is that it is another Gaussian, which in image space is just a small, soft blob. Convolving the image with a small soft blob is precisely what a gentle blur is. Notice that this is the very same theorem that explained the ghost copies back in Chapter 3. There it explained ghosts, and here it explains blur. One theorem accounts for every artifact we have seen so far.

4.3 Thermal noise: why real scans are grainy

The Concept in Plain English

Let us now turn to the second topic of this milestone, which is noise. The MRI signal is a faint radio whisper, and it is measured by an antenna. The trouble is that every warm object is constantly emitting a random electromagnetic crackle of its own. The patient’s body does this, and so do the receiver electronics. We call this **thermal noise**, and it adds itself to every single k -space sample at the moment of acquisition. There is an important detail here. The measurement is complex, meaning that it has both a magnitude and a phase, and so the noise strikes *both* the real channel and the imaginary channel, and it does so independently in each one. In symbols, that looks like this:

$$K_{\text{noisy}}[u, v] = K[u, v] + \mathcal{N}(0, \sigma_n^2) + i\mathcal{N}(0, \sigma_n^2), \quad (4.3)$$

where $\mathcal{N}(0, \sigma_n^2)$ stands for a random draw from a Gaussian, which is the familiar bell

curve, with a mean of zero and a standard deviation of σ_n . Now remember that the Fourier transform spreads every single k -space sample across the whole image. Because of that, the noise does not stay in one place. It lands *everywhere*, and what you see is a uniform grain spread over the entire reconstruction.

There are two facts about this noise that are worth knowing, and they are also worth demonstrating in your app:

- **Noise must be added in k -space, not to the image.** The reason is simply that k -space is where the noise physically arises. There is a subtle consequence of getting this right. When we take the magnitude of the reconstruction at the end, the dark regions of the image show a characteristic “floor” of noise that is not Gaussian at all. Here is why. A magnitude can never be negative, so in the black areas the noise has nowhere to go but upward, and it piles up above zero. The distribution this produces is called the Rician distribution. If you were to add noise directly to the final image instead, you would miss this effect entirely.
- **Noise and undersampling compound each other.** When you acquire fewer samples, there is less signal averaging going on, and so accelerated scans are noisier in practice. You will be able to see this for yourself on your metrics dashboard in Milestone 5, where the PSNR will fall as you move either slider, and fall further when you move both at once.

4.4 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in `filters.py`. It is deliberately a light one. Think of it as a breather between the sampling milestone you have just finished and the gridding milestone that comes next.

A. Centered coordinate grids (build once, reuse everywhere)

- Write one helper that returns two matrices, U and V . For every entry, U holds that entry’s horizontal offset from the center of the matrix, and V holds its vertical offset. You already needed exactly this when you built the circular mask in Milestone 2, so please reuse that helper rather than writing it a second time.

B. The window generators

- **Hamming:** first build the one-dimensional raised-cosine profile along one axis, then build it along the other axis, and then form the two-dimensional window as the outer product of the two. The only parameter this function needs is the matrix size.
- **Gaussian:** evaluate the radial Gaussian formula on the U, V grids you built in step A. This function is parameterized by σ . One tip here is to express σ as a fraction of the matrix half-width rather than as a raw number of pixels, so that the slider feels natural no matter what size the image is.
- Both functions must return a matrix of the same shape as k -space. The values should lie in $[0, 1]$, and the maximum should sit at the center.

C. The noise injector

- Write a function that takes a complex k -space matrix and a noise level σ_n . Inside it, draw one Gaussian random matrix for the real part and a second, *independent* one for the imaginary part. Combine those two into a single complex matrix and add it to the input.

- Accept an optional seed, so that the same call can be reproduced exactly when you need it to be.
- Scale σ_n relative to the mean magnitude of k -space. For example, the slider could set a percentage. The point of doing this is that “noise level 0.05” then means the same thing across different input images.

D. Pipeline placement

- The order of operations matters here, and it should mirror what physically happens in the scanner. Start from the full k -space. *First* add the noise, because the scanner receives noisy data. *Then* apply the sampling mask, because the scanner only keeps the lines it actually acquired. *Then* apply the apodization window, because that is a processing choice we make after acquisition is over. Only then do you reconstruct.

E. The demonstration experiment

- Reconstruct the phantom through a tight circular mask, which you already have from Milestone 2, with no window applied. Zoom into a sharp boundary and capture the ripples you see there.
- Now repeat the same reconstruction with the Hamming window applied. Save the two images side by side. You should see that the ringing is gone and that the edge has become slightly softer.
- Finally, reconstruct with three different noise levels, for example σ_n at 1%, 5%, and 15% of the mean magnitude, and save the progression from clean to grainy.

How to Test It Before Moving On

- **Window shape test:** both windows must equal exactly 1.0 at the center entry, and they must decrease monotonically as you move outward along any axis. In addition, the Gaussian must be radially symmetric, which means its value depends only on the distance from the center.
- **Ringing test:** plot a single row of the reconstruction where it crosses a sharp edge of the phantom, once with the window and once without. The un-windowed curve must visibly overshoot and oscillate. The windowed curve must not.
- **Noise statistics test:** zero out the image entirely, inject noise, and reconstruct. When you double σ_n , the standard deviation of the result should double as well, that is, it should scale linearly.
- **Determinism test:** running the injector twice with the same seed must produce the identical noisy reconstruction both times.

Self-Check: Can You Answer These?

1. What causes Gibbs ringing, and near which image features does it appear?
2. Why does adding *more* frequencies never eliminate Gibbs ringing, only narrow it?
3. Explain apodization using the fade-out analogy, then state precisely what is multiplied with what.
4. What is the fundamental trade-off of every apodization window? Which slider in your app exposes it?
5. Use the convolution theorem to explain why a Gaussian window in k -space blurs the

image.

6. Why is MRI noise added to the *complex k-space* rather than to the final image, and what visible difference does that choice make in dark image regions?
7. Why does the Hamming window's floor value of 0.08 (rather than 0) represent a design compromise?

Milestone 4 — Radial & Spiral Trajectories, Density Compensation, and Gridding

Let us be honest with each other before we start: this is the most mathematically demanding milestone in the whole project. Please do not begin it until Milestones 1, 2 and 3 are finished and every one of their tests passes, because everything here builds directly on the Cartesian k -space and the inverse-transform function you wrote back in Milestone 1. Here is the good news, though. When you look closely, this milestone breaks apart into three small ideas that do not depend on each other very much. The first idea is *drawing paths* through k -space. The second idea is *re-weighting points* so that crowded regions do not dominate. The third idea is *snapping points onto a grid* so that the FFT can accept them. Each of these is genuinely simple once you look at it on its own, and we are going to take them one at a time.

5.1 Why scanners abandon straight lines

The Concept in Plain English

Think back to how we sampled k -space in the earlier milestones. Cartesian scanning reads k -space the way you read a book, going row by row, and left to right within each row. Notice, however, that nothing in the physics actually requires straight rows. The scanner's magnetic gradients can steer the sampling point along *any* smooth path through k -space that we care to design. Two alternative families of paths dominate real-world practice, and we are going to build both of them.

- **Radial:** here the scanner measures along straight spokes that all pass through the center of k -space. You can picture the spokes of a wheel, or the hands of a clock sweeping around the dial. Because every single spoke passes through the center, the low frequencies, which you will remember carry the overall contrast of the image, get re-measured on every spoke. This has a very practical consequence. If the patient moves during the scan, there is no single “stripe” of k -space that gets ruined, because no one spoke owns a region of k -space by itself. The motion errors instead smear out into mild streaks rather than turning into catastrophic ghost copies. This is exactly why radial imaging is the method of choice for children and for cardiac patients, who cannot hold still.
- **Spiral:** here the scanner follows one continuous corkscrew path that starts at the center and winds its way outward. Because the path is smooth, the gradients never have to slam from one extreme to the other, and that means a spiral can cover the whole of k -space extremely quickly. This is why spirals are popular in functional MRI, where speed is everything.

There is a price to pay for these benefits, and it is the reason this milestone exists. The samples collected along a spoke or a spiral no longer land on the neat integer grid that the FFT demands as its input. Fixing that mismatch between where the scanner sampled and where the FFT wants its data is the whole engineering content of this milestone.

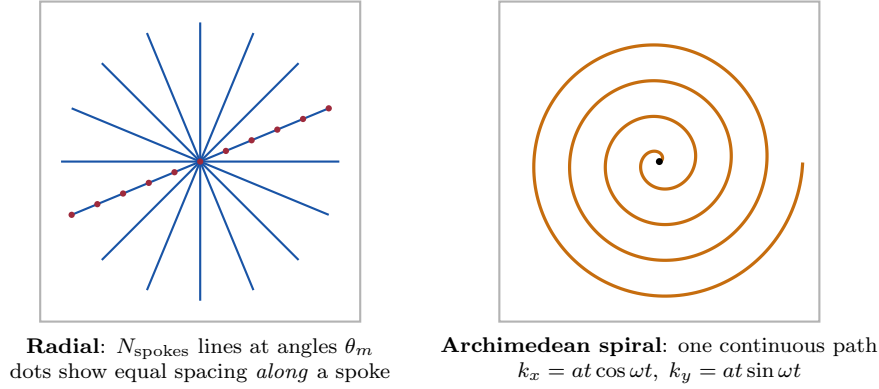


Figure 5.1: The two non-Cartesian trajectories. Neither lands on the integer grid the FFT requires.

5.2 Drawing the paths: it is only trigonometry

Radial. Let us start with the radial trajectory, because it is the simpler of the two. A single spoke is just a straight line that passes through the origin of k -space at some angle θ . To build a full radial trajectory, we choose N_{spokes} angles and spread them evenly over half a turn. You might wonder why half a turn is enough. The reason is that a spoke at angle θ and a spoke at angle $\theta + 180^\circ$ trace out exactly the same line through the origin, so covering the second half of the circle would only repeat lines we already have. So the angles are

$$\theta_m = \frac{m\pi}{N_{\text{spokes}}}, \quad m = 0, 1, \dots, N_{\text{spokes}} - 1. \quad (5.1)$$

Now, along each spoke, we walk a radius variable r from $-k_{\text{max}}$ all the way to $+k_{\text{max}}$ in equal steps, and at each step we convert the polar description (a radius and an angle) into ordinary Cartesian coordinates. This is secondary-school trigonometry and nothing more:

$$k_x(r, \theta_m) = r \cos \theta_m, \quad k_y(r, \theta_m) = r \sin \theta_m. \quad (5.2)$$

Spiral. The spiral we use is called an Archimedean spiral, and it has a very simple rule: its radius grows linearly while its angle turns at a steady rate. To describe it we introduce a time-like parameter t that increases as we move along the path, and then the coordinates are

$$k_x(t) = at \cos(\omega t), \quad k_y(t) = at \sin(\omega t), \quad (5.3)$$

where ω sets how fast the spiral turns and a sets how fast its radius grows. In practice we do not pick a freely. We choose it so that the spiral just reaches k_{max} at the final value of t , which means it fills the same region of k -space that our Cartesian data covered.

Where do the sample values come from in a simulation? This is a question that trips up many students, so let us spell it out. On a real scanner, the hardware physically measures the signal at each point along the trajectory. We do not have a scanner. What we do have is the full Cartesian k -space matrix from Milestone 1, and so we **interpolate** it, which simply means looking up the in-between values, at each coordinate on our trajectory. The effect is exactly as if the scanner had sampled at those positions. We call this the “forward interpolation” step. It exists only because we are simulating rather than scanning, and for that very reason it is easy to forget, so keep it in mind.

5.3 The density problem: spokes crowd the center

Analogy

Picture a bicycle wheel. Near the hub, the spokes are packed so tightly that they almost touch each other. Out at the rim, the very same spokes are spread far apart. Now here is why this matters to us. Radial k -space sampling has exactly this geometry. The center of k -space gets sampled by *every* spoke, because every spoke passes through it, while each little patch of the periphery is visited by only one spoke. Suppose you naively add all of the samples onto a grid without thinking about this. The center of k -space, which holds the low frequencies and therefore the contrast of the image, would be counted dozens of times over. The reconstruction then comes out as a foggy, glowing, contrast-crushed mess. It is technically still an image, but it is drowning in low-frequency glare.

The cure for this problem is called **density compensation**. Before we do any gridding, we multiply each sample by a weight, and you can think of that weight as answering the question “how much area of k -space do *you* alone represent?” A sample sitting in crowded territory near the center represents only a tiny sliver of area, so it receives a small weight. A lonely sample out at the periphery represents a large patch of area all by itself, so it receives a large weight. For radial trajectories there is a lovely simplification. The crowding is exactly inversely proportional to the radius, and so the weight that undoes it is simply proportional to the radius. This weighting has a famous name, the **ramp filter**, and it is written

$$w(r) = |r| = \sqrt{k_x^2 + k_y^2}. \quad (5.4)$$

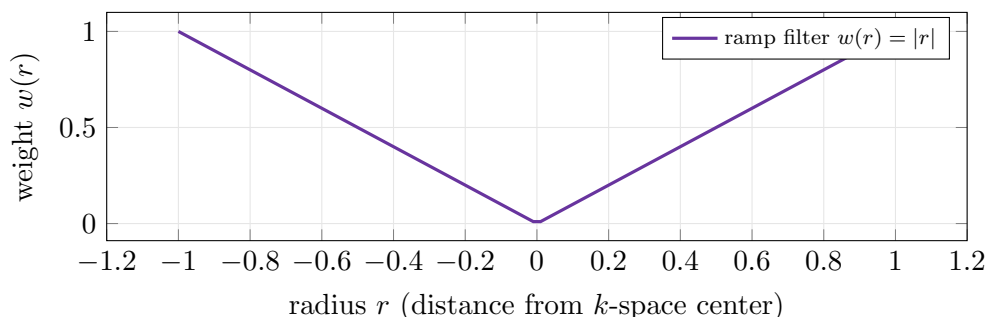


Figure 5.2: The radial density compensation function. Center samples (massively over-represented) are weighted toward zero; edge samples (sparse) are weighted up. Without this, reconstructions are drowned in low-frequency glare.

Why This Matters

Let me tell you about the most satisfying moment in this milestone. First, reconstruct some radial data *without* any density compensation and look at the result. Then flip the compensation on and reconstruct again. You will watch the image snap, instantly, from a blurry glowing fog into a properly contrasted picture. I strongly encourage you to put a checkbox for exactly this in your GUI, because it is the best five-second demonstration in the entire application.

5.4 Gridding: getting scattered points back onto the FFT's grid

The Concept in Plain English

It helps to think of the inverse FFT as a machine with a very rigid input socket. It will accept only a full rectangular matrix of values sitting at integer positions, and nothing else. Our weighted trajectory samples, on the other hand, are scattered at arbitrary real-valued coordinates, something like $(37.42, -101.77)$. The FFT simply cannot take those directly. **Gridding** is the adapter that sits between the two. The idea is that every scattered sample deposits its value onto the integer grid cells that are nearest to it. Once all of the samples have been deposited in this way, we are left with an ordinary matrix, and from there we can proceed exactly as we did in Milestone 1.

There are three strategies for doing this deposit, and we will go through them in increasing order of quality.

1. **Nearest neighbor.** In this strategy each sample dumps its entire value into the single closest grid cell, which you find by rounding both coordinates to the nearest integer. This is crude, and it is worth understanding why. If two samples happen to round to the same cell, they collide. Meanwhile, any cell that no sample rounds to stays empty. Even so, it is perfectly fine as a first prototype.
2. **Bilinear spreading.** Here each sample splits its value among the *four* grid cells that surround it, and it does so in proportion to how close it is to each one. Suppose a sample sits at fractional position (α, β) inside its cell. Then the four corners receive the weights

$$(1-\alpha)(1-\beta), \quad \alpha(1-\beta), \quad (1-\alpha)\beta, \quad \alpha\beta, \quad (5.5)$$

and notice that these four numbers always add up to exactly 1. That is an important property, because it means no signal is created and no signal is lost during the spreading. The results are much smoother than nearest neighbor.

3. **Kernel convolution (e.g. Kaiser–Bessel).** In this strategy each sample spreads its value over a small neighborhood of cells, typically 4×4 or more, and the spreading is shaped by a carefully designed bump function. This is what professional reconstruction packages do. In this project it is an optional stretch goal, and you should know that bilinear spreading is fully sufficient for the demonstration.

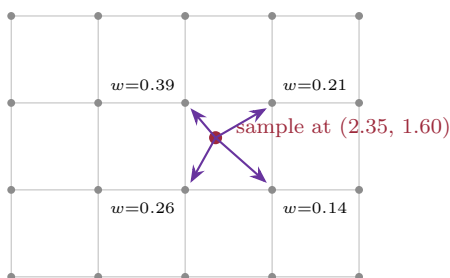


Figure 5.3: Bilinear gridding: one scattered sample (red) splits its value among its four surrounding integer grid cells, weighted by closeness. The four weights sum to exactly 1.

There is one more bookkeeping detail that you must not skip. Alongside the data grid, keep a second matrix of the same size, which we call the **weight-accumulation matrix**. Every time a sample deposits a weight w into some cell of the data grid, also add that same w into the same cell of the weight matrix. When all the samples have been deposited, divide the data grid by the weight grid, wherever the weight is nonzero. Here is why this matters. Some cells will happen to

receive contributions from many samples, and others from only a few. Without this final division, the cells that received many samples would end up artificially bright, simply because more things were added into them.

5.5 What the artifacts look like now

You might expect undersampled radial data, meaning data with too few spokes, to produce the same clean ghost copies that we saw with Cartesian undersampling. It does *not*, and the reason is worth understanding. The point spread function of a spoke pattern is a star shape rather than a row of clean copies. As a result, the artifact takes the form of **streaks**, which are fine straight lines radiating across the image, with one family of streaks for each missing angle. Spiral undersampling behaves differently again, and it produces swirling, curved artifacts. When your application is finished, the signature experiment will be to put these three artifact families side by side, the ghosts from Cartesian, the streaks from radial, and the swirls from spiral, all at a matched acceleration, so that you can compare them directly.

5.6 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in two files. The first is `trajectories.py`, which is responsible for the paths and the density weights. The second is `gridding.py`, which is responsible for scattering samples onto the grid. Please keep these two strictly separate. The trajectory module must know nothing at all about grids, and the gridding module must work for *any* list of coordinates you hand it, not just the ones your trajectory module happens to produce.

A. Trajectory generation (`trajectories.py`)

- For the radial trajectory, you are given N_{spokes} and the number of samples per spoke, and you need to produce the list of (k_x, k_y) coordinates. To do that, compute the evenly spaced angles over $[0, \pi)$, walk r symmetrically from $-k_{\text{max}}$ to $+k_{\text{max}}$, and apply the cosine and sine conversion we wrote down earlier. Return the coordinates as flat arrays.
- For the spiral trajectory, you are given a total number of samples and a number of turns. Generate the parameter t , apply the Archimedean formulas, and choose the growth constant so that the final radius comes out equal to k_{max} .
- For the density weights in the radial case, return $w = \sqrt{k_x^2 + k_y^2}$ for every sample. For the spiral, using that same radial ramp is an acceptable approximation at this project's level, but you should say clearly in your report that it is an approximation.
- Add a quick visual check as well. A scatter plot of the generated coordinates, colored by weight, will make any mistake in the math visible instantly.

B. Simulated acquisition (the forward direction)

- Write the “pretend scanner”. It is given the full Cartesian k -space from Milestone 1 together with a list of trajectory coordinates, and its job is to interpolate the complex k -space values at those exact coordinates. Use a library interpolator for this. Be aware that complex data may require you to interpolate the real and imaginary parts separately.

C. The gridding engine (`gridding.py`)

- **Prototype first** using the library scattered-interpolation routine, the `griddata`-style one. Hand it the coordinates, the density-weighted values, and the target integer grid, and ask it for linear interpolation with zero fill outside the sampled region. This approach gets you a working end-to-end radial reconstruction in a single afternoon.
- **Then** implement your own bilinear spreader, because this is the part that is worth marks. Allocate a zero complex matrix for the data and a zero weight matrix alongside it. For each sample, find the four surrounding integer cells, compute the four closeness weights, and deposit both the weighted value and the weight itself. Finally, divide the data by the weights wherever the weights are nonzero.
- Guard the edges carefully. A sample whose neighborhood falls outside the matrix must be clipped or skipped. It must never be allowed to wrap around to the other side.
- After gridding, what you are holding is ordinary centered k -space, so reconstruct it with the exact same inverse-transform function you wrote in Milestone 1.

D. The signature experiments

- The density-compensation reveal. Take the same radial data and reconstruct it once with the weights off and once with the weights on, and save both images.
- The spoke-count sweep. Reconstruct at, say, 16, 32, 64, 128 and 256 spokes, and watch the streaks fade as the number of spokes increases.
- The artifact-family comparison. Put the Cartesian $R=4$ ghosts, the radial streaks and the spiral swirls side by side, at matched sample counts.

How to Test It Before Moving On

- **Coordinate sanity:** the samples of every radial spoke must lie on a straight line through the origin. The maximum radius must equal $k_{\max}$. Your scatter plot must look like Figure 5.1.
- **Weight conservation:** in your bilinear spreader, the four weights for any sample must add up to 1, within floating-point error.
- **Dense-limit test:** when you use very many spokes, for example $\geq 2N$, and turn density compensation on, the radial reconstruction must closely match the fully sampled Cartesian reconstruction. A small residual error is expected from the interpolation, but the two images should be visually indistinguishable.
- **Glare test:** switching density compensation *off* must produce the characteristic foggy low-frequency glare. If nothing changes when you flip it, then your weights are not actually being applied.
- **Streak test:** at 32 spokes you must see straight streaks radiating across the image, and you must not see vertical ghost copies. If you do see ghost copies, that means you accidentally ran a Cartesian mask.

Self-Check: Can You Answer These?

1. Give two clinical reasons a scanner might prefer radial acquisition over Cartesian.
2. Why can radial angles be confined to $[0, \pi)$ instead of a full turn?
3. Explain, with the bicycle-wheel picture, why uncompensated radial data reconstructs

with crushing low-frequency glare.

4. Why is the radial density weight exactly proportional to $|r|$?
5. Why can the inverse FFT not be applied directly to trajectory samples? What property of its input does it demand?
6. Walk through what happens to a single sample at coordinates (10.3, 20.8) in nearest-neighbor gridding vs. bilinear gridding.
7. Why must you accumulate a separate weight matrix and divide by it at the end?
8. Why do too-few spokes produce *streaks* while skipped Cartesian rows produce *ghost copies*? (Answer via the shape of each pattern's PSF.)

Milestone 5 — Measuring Image Quality: MSE, PSNR, and SSIM

6.1 From “it looks worse” to actual numbers

The Concept in Plain English

Think back over everything we have built so far. Every one of the previous milestones takes a perfectly good image and *damages* it in some controlled way. We skipped k-space rows, we faded out high frequencies, we added noise. In this milestone we stop and ask a question we have been avoiding: *by how much, exactly, did we damage it?* Up to now the simulator has only shown you pictures, and pictures are wonderful for building intuition. But a simulator that also shows you **numbers** teaches something closer to science. The reason is that a number can be plotted against the acceleration slider, a number can be compared across two sampling strategies, and a number can be quoted in your report. A picture cannot do any of those things on its own. So in this chapter we are going to add three standard metrics, and each of them answers a subtly different question about image quality. On top of that we are going to add one visual tool, the error heatmap, whose job is to show you *where* in the image the damage lives.

Before we start, let us fix our notation so that nothing is ambiguous later. Throughout this chapter, f stands for the ground-truth original image, the clean one we started with. The symbol $\hat{f}$ stands for the reconstruction, which is whatever came out of the pipeline after the undersampling or filtering. Both of these images have the same size, $N \times M$ pixels, and both of them are normalized so that every pixel value lies in $[0, 1]$.

6.2 MSE: the average squared mistake

$$\text{MSE} = \frac{1}{NM} \sum_{m=0}^{N-1} \sum_{n=0}^{M-1} |f[m, n] - \hat{f}[m, n]|^2. \quad (6.1)$$

Let us read this formula literally, one piece at a time. At every single pixel, you take the difference between the original value and the reconstructed value. That difference is the error at that pixel. Then you square it. There are two reasons for squaring. The first reason is that squaring makes every error positive, so an error of $+0.1$ at one pixel and an error of -0.1 at the next pixel cannot cancel each other out and pretend that nothing went wrong. The second reason is that squaring punishes big errors much more than small ones, because a mistake twice as large counts four times as much. Finally, you add all of those squared errors up and divide by the total number of pixels, which gives you the average. That average is the **Mean Squared Error**, or MSE.

MSE is simple to write, fast to compute, and it is the building block we will use to construct PSNR in the next section. But you should know its weakness from the start. MSE is *blind to structure*. Here is what that means in practice. Suppose you shift an image sideways by a single pixel. A human looking at the two images would barely notice any difference at all. MSE, on the other hand, can report a large error, because every edge in the picture now lands on a different pixel than before. Now imagine the opposite situation. A smooth drift in brightness across the

whole image looks harmless to a person, and ugly localized ghosting looks terrible to a person, and yet the two can produce exactly the same MSE. MSE only knows about the size of the mistakes. It knows nothing about what the mistakes look like.

6.3 PSNR: MSE on a decibel scale

$$\text{PSNR} = 10 \log_{10} \left(\frac{\max(f)^2}{\text{MSE}} \right) \quad [\text{dB}]. \quad (6.2)$$

The **Peak Signal-to-Noise Ratio**, or PSNR, does not measure anything new. It takes the MSE we just computed and repackages it as a ratio. The question it asks is this: how large is the strongest possible signal in the image, compared to the average error we made? The numerator is the square of the maximum value the image can take, and the denominator is the MSE. We then express that ratio in decibels by taking the base-ten logarithm and multiplying by ten. The reason we use a logarithm is that it matches how quality differences actually feel to a viewer. Each step of roughly 6 dB corresponds to halving the error amplitude, so equal steps on the decibel scale feel like equal steps in perceived quality.

It helps to have some anchors in mind when you read PSNR values off your dashboard. When the PSNR is above roughly 40 dB, the reconstruction looks essentially perfect to the eye. When it sits somewhere between 30 and 40 dB, you will see mild but visible degradation. When it drops below roughly 25 dB, the damage is obvious to anyone. Pay attention to the direction here, because it is the opposite of MSE. For PSNR, **higher is better**. There is one more thing to notice. For a perfect reconstruction the MSE is zero, and dividing by zero makes the PSNR infinite. That is mathematically correct, but your code must handle that division as a special case rather than letting it crash or produce a warning.

6.4 SSIM: asking what a human would say

Both MSE and PSNR compare the two images one pixel at a time, with each pixel treated in isolation. Human vision does not work that way. When you look at a picture, your eye compares *neighborhoods*. It notices the local brightness of a region, the local contrast of that region, and the local structure, meaning the pattern of light and dark inside it. The **Structural Similarity Index**, which we abbreviate as SSIM, was designed to formalize exactly those three comparisons. Here is how it works. It slides a small window across both images at the same time. Inside each window it computes the local means μ_f and $\mu_{\hat{f}}$, the local variances σ_f^2 and $\sigma_{\hat{f}}^2$, and the local covariance $\sigma_{f\hat{f}}$. The covariance is the interesting one. It asks whether the two windows brighten and darken *together*, which is really a question about whether they share the same pattern. Once it has all of those quantities, it combines them like this:

$$\text{SSIM}(f, \hat{f}) = \frac{(2\mu_f\mu_{\hat{f}} + C_1)(2\sigma_{f\hat{f}} + C_2)}{(\mu_f^2 + \mu_{\hat{f}}^2 + C_1)(\sigma_f^2 + \sigma_{\hat{f}}^2 + C_2)}. \quad (6.3)$$

- The first factor is the one that compares **luminance**. If you look at it closely, you will see that it equals exactly 1 when the two local mean brightnesses agree, and it drops below 1 as they drift apart.
- The second factor is the one that compares **contrast and structure**, and it does so through the covariance. This factor is large only when the two windows vary *in the same pattern*. If one window has a bright stripe where the other has a dark stripe, the covariance goes negative and this factor collapses.
- The constants C_1 and C_2 are tiny numbers whose only job is to prevent division by zero in flat regions of the image, where the means or variances might be exactly zero. You should

think of them as stabilizers. They are not part of the physics, and they do not change the answer in any region that has real content.

- The final SSIM score is the average over all of the windows, and it lands somewhere in $[-1, 1]$. A score of 1 means the two images are structurally identical. A score near 0 means they are structurally unrelated.

Why This Matters

You might reasonably ask why we bother carrying three metrics when one would be simpler. The answer is that the three metrics *disagree* with each other, and those disagreements are exactly where the information is. Let me give you two examples. Suppose you apply a mild blur to the whole image. The structure is mostly still there, so SSIM stays reasonably high, but the PSNR drops sharply because every pixel has shifted a little. Now suppose instead you add strong noise. This time the structure is destroyed, so SSIM falls hard, while the average error stays moderate and the PSNR does not fall as much. When your dashboard shows all three numbers side by side, the user learns more than the fact *that* quality dropped. The user learns *what kind* of quality dropped. That is precisely the pedagogical point of building this simulator in the first place.

6.5 The error heatmap: seeing *where* it went wrong

The three metrics each boil the whole image down to a single number. That is useful, but it throws away the location of the damage. The fourth panel of your GUI is there to bring that location back. It displays the absolute difference image

$$E[m, n] = |f[m, n] - \hat{f}[m, n]| \quad (6.4)$$

and renders it with a colormap. Dark regions of the heatmap are places where the reconstruction was accurate, and bright or hot regions are places where it was wrong. Once you start using this panel you will notice that every artifact family paints its own recognizable signature. Ghosting from the uniform undersampling milestone shows up as displaced copies of the anatomy’s outline. Gibbs ringing shows up as bright fringes hugging every sharp edge. Radial undersampling shows up as a fan of streaks spreading out across the image. Noise shows up as a uniform speckle with no particular shape. Because of this, the heatmap is also the app’s best debugging tool. When something in the pipeline misbehaves, one glance at the heatmap usually tells you *which* stage was responsible.

6.6 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

All of this work lives in the file `metrics.py`. I want to reassure you up front that this is the easiest module in the whole project. If anything slipped in an earlier milestone, this is a good week to catch up on it.

- Start by writing the MSE function directly from its formula. You take the element-wise difference between the two arrays, you square the magnitudes, and you take the mean. That is two lines of actual logic.
- Next, write PSNR on top of the MSE function you just wrote. Remember the perfect-reconstruction case we discussed, and handle it explicitly. If the MSE comes out as zero, return infinity, or display “ ∞ dB” in the GUI, rather than letting the code

divide by zero.

- Please do *not* hand-implement SSIM. The sliding-window bookkeeping is easy to get subtly wrong, and `scikit-image` already ships a reference implementation that has been checked by many people. Call that function instead. One detail matters a great deal here. You must pass the correct `data_range` argument, which is the difference between the maximum and minimum values your images can possibly take. The reason this is required is that your images are floating-point arrays in $[0, 1]$, and the library would otherwise assume they are 8-bit integers that go up to 255.
- Then write the error-map function. It simply computes the absolute difference matrix and returns it as it is. Let the display layer decide on the colormap. When you get to that display layer, choose a perceptually ordered colormap such as `viridis` or `inferno`, and stay away from the rainbow “jet” colormap. The problem with jet is that it fabricates visual edges at places in the data where no edge actually exists.
- Bundle everything into one convenience call. It should take the pair $(f, \hat{f})$ and return all three numbers plus the error map, packed into a single dictionary. This becomes the one call the GUI makes after every reconstruction, so it is worth making it tidy.
- **Consistency guard:** the metrics only mean something if both images are on the same scale. So before you compute anything, assert that both inputs lie in $[0, 1]$, or else normalize both of them in exactly the same way. I stress this because a silent scale mismatch is the classic source of nonsense PSNR values, and it is very hard to spot after the fact.

How to Test It Before Moving On

- **Identity test:** compare any image with itself. The result must be $\text{MSE} = 0$, $\text{PSNR} = \infty$, and $\text{SSIM} = 1.0$, and all three must be exact.
- **Known-value test:** build an all-black image, where every pixel is 0.0, and an all-mid-gray image, where every pixel is 0.5. Compare them. The MSE must come out as exactly 0.25, because every pixel has an error of one half and the square of one half is one quarter.
- **Monotonicity test:** reconstruct the phantom with the uniform mask at $R = 1, 2, 4, 8$. As R grows, both PSNR and SSIM must decrease monotonically. If a higher R ever scores *better* than a lower one, something in your pipeline is broken.
- **Sensitivity ordering:** take two copies of the same image. Add slight blur to one copy, and add strong noise to the other, adjusting the amounts so that the two copies have matched MSE. Then verify that SSIM ranks the blurred copy higher. If it does, you have confirmed that SSIM really is capturing structure and not just the size of the error.

Self-Check: Can You Answer These?

1. Why does MSE square the pixel differences instead of just averaging them?
2. What question does PSNR answer that raw MSE does not, and why is a logarithmic (decibel) scale appropriate?
3. Give a concrete example where two distortions have equal MSE but very different visual quality. Which metric detects the difference?

4. What three local quantities does SSIM compare, and what role do the constants C_1, C_2 play?
5. Why is passing the correct `data_range` essential when computing SSIM on floating-point images?
6. You look at an error heatmap and see bright fringes tracing every sharp edge. Which artifact is this, and which milestone's tool fixes it?
7. Which direction is “better” for each metric, and what are the best-possible values of all three?

Milestone 6 — The Interactive GUI and Application Architecture

7.1 From engine to instrument

The Concept in Plain English

Let us pause for a moment and look at what you have actually built so far. Everything in Milestones 1 through 5 is an **engine**. By that we mean a collection of functions inside the `mri/` package that you call from a script, wait for, and then inspect by printing numbers or saving images. That is exactly the right way to build the mathematics, but it is not yet something a person can play with. In this milestone we are going to wrap that engine in an **instrument**. The instrument is a window, written in `app.py`, where you drag a slider, the whole pipeline runs again behind the scenes, and four picture panels repaint themselves right away.

Here is why this milestone matters so much for a teaching project. Imagine a student who has read about aliasing and ghosting but has never really felt it. Now imagine that student dragging the acceleration slider and watching, with their own eyes, as ghost copies crawl into the reconstructed image while the PSNR number in the corner falls. At that moment the abstract mathematics turns into physical intuition. The whole teaching power of the project lives in that moment, and this chapter is about making it happen.

There is one big idea you need before we start, and it is about how desktop programs run. Every script you have written until now runs from top to bottom: line one, then line two, and so on until the end. A desktop GUI does not work that way. A GUI is **event-driven**. The program builds its window, and then it sits in an idle loop, doing nothing at all, simply waiting for something to happen. An **event** is one of those happenings, for example a slider being moved or a dropdown being changed. For each kind of event you register a function in advance, and the program calls that function whenever the event occurs. That registered function is called a **callback**, or in the Qt world a **slot**. So your entire job in this milestone comes down to three things. You build the layout once. You connect each control to one callback. You make that callback run the pipeline and refresh the panels.

7.2 The layout

Take a look at Figure 7.1 and let us walk through it together. The window has four large picture areas, which we will call panes, and a narrow column of controls down the right-hand side. Each pane is a plotting canvas that lives inside the window. You do not need to invent this yourself, because Matplotlib already provides a Qt widget that is designed for exactly this purpose. Pane 1 shows the original image, which is the ground truth we are trying to recover. Pane 2 shows the k -space data, drawn as a log magnitude so that you can actually see it, with the sampling mask or the trajectory drawn on top. Pane 3 shows the reconstructed image, and Pane 4 shows either the error heatmap or the point spread function, depending on a checkbox.

Notice that the mask overlay in Pane 2 deserves special attention, because it is what visually links cause to effect. The user can see exactly which k -space lines survived the subsampling, and

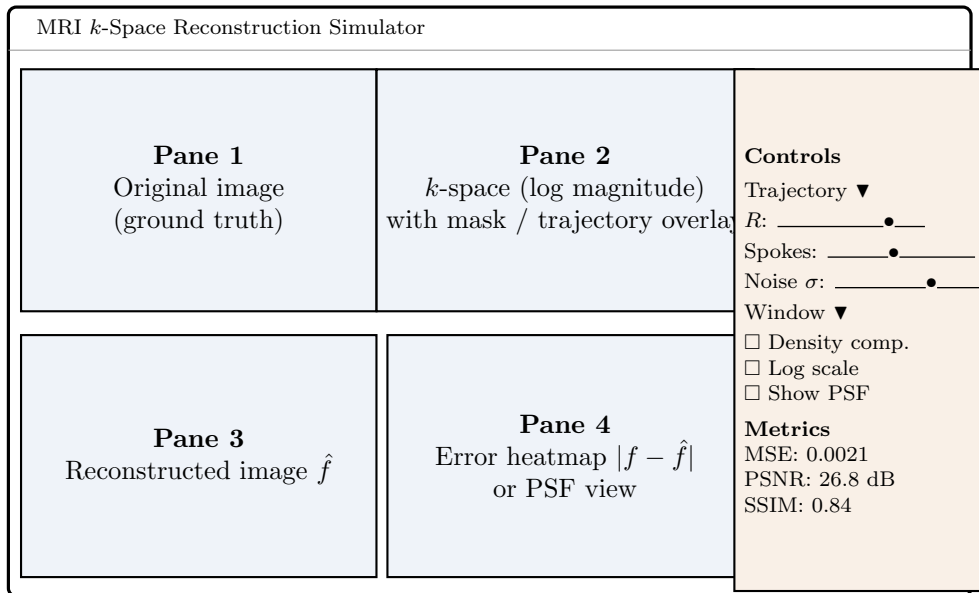


Figure 7.1: The dashboard wireframe: four synchronized viewing panes plus a control column with live metric readouts.

right next to that picture, in Pane 3, they can see the damage that the missing lines caused. Putting the cause and the effect side by side is the whole point of the layout.

7.3 The one problem that ruins naive GUIs: freezing

Analogy

Here is an analogy that will carry you through the hardest part of this milestone. Think of a GUI as an office with a single receptionist. That receptionist is the **main thread** of the program. The receptionist's job is to answer events as they come in. That means repainting the panels, responding to the mouse as you drag a slider, and generally keeping the window alive and awake.

Now imagine what happens if the receptionist personally walks away from the front desk to run your reconstruction. Gridding alone can take a noticeable fraction of a second, which is a long time for a front desk to be empty. While the receptionist is away, nobody is answering events. The window stops repainting. The slider freezes in the middle of your drag. After a moment the operating system notices that nobody is home and grays the whole application out with the label "Not Responding."

The fix is obvious once you say it out loud. The receptionist should *delegate*. Any heavy computation gets handed to a helper in a back room, which in programming terms is a background **worker thread**. The receptionist stays at the desk and keeps answering events the whole time. When the worker in the back room finishes, it sends a short message up to the front desk. In Qt that message is called a **signal**. The only work that happens at the front desk is the final, lightweight repaint of the panels.

There are two rules that keep a threaded GUI sane, and you should treat both of them as iron laws rather than as suggestions. Let us go through each one and explain why it exists.

1. **Only the main thread touches widgets.** The worker thread is allowed to compute pure data, meaning matrices and numbers, and nothing else. It must never reach out and change a widget directly. Instead, the worker communicates its results exclusively through the

signal mechanism, because the signal mechanism is designed to cross the boundary between threads safely. If you break this rule the program may seem to work for a while and then crash in ways that are very hard to reproduce.

2. **Stale results must be discarded.** Think about what happens when a user drags a slider. The slider fires dozens of change events every second. If you queue up a full pipeline run for every single one of those events, the computer falls further and further behind. You have two sensible options here. One option is to drop any new requests while a worker is busy and then run the pipeline once more at the final slider value. The other option is to cancel the worker that is currently in flight. If you do neither, the display ends up lagging behind the slider by many seconds, patiently playing back obsolete frames that nobody wants to see anymore.

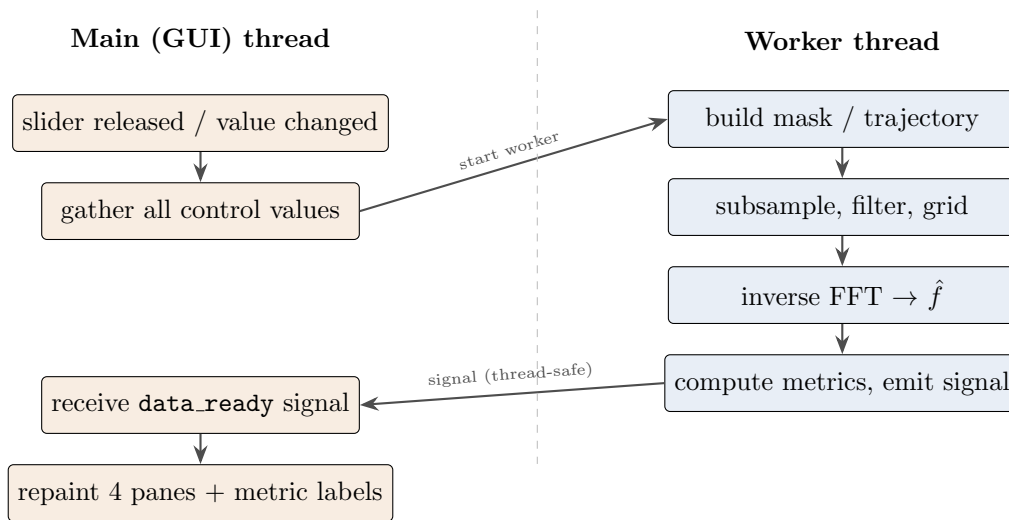


Figure 7.2: The asynchronous control flow. The main thread never computes; the worker thread never touches widgets. They communicate only via signals.

Figure 7.2 draws the whole story as two lanes. Follow the left-hand lane first. The main thread notices that a slider has moved, it gathers the current value of every control, and then it starts the worker and goes straight back to waiting for events. Now follow the right-hand lane. The worker builds the mask or trajectory, subsamples, filters, and grids the data, runs the inverse FFT to get the reconstruction, computes the metrics, and finally emits a signal. That signal is the only arrow that crosses the dashed line back to the main thread. When the main thread receives it, all it has to do is repaint the four panes and update the metric labels.

7.4 The architecture that keeps six weeks of code sane

By the time you reach this milestone you will have several weeks of code, and it is very easy for a GUI project to collapse into one enormous file where everything depends on everything else. To prevent that, the application is organised into three layers, and each layer is only allowed to know about certain things. Let us go through the three layers one at a time.

- The first layer is the **View**. This is the GUI file, `app.py`, and it owns the windows, the panes, and the sliders. The important thing to understand is that the View knows nothing at all about Fourier transforms. Its only job is to collect the current settings from the controls and to display whatever matrices it is handed.
- The second layer is the **Pipeline engine**, which is one coordinating class. It receives a bundle of settings, meaning the trajectory type, the acceleration factor R , the noise level

σ_n , the window choice, and the state of each checkbox. It then calls your Milestone 1 to 5 modules in the correct order and hands back a bundle of results, which is four display matrices plus three metrics. This class is the *only* code in the whole project that knows the order of operations of the pipeline.

- The third layer is made of the **Engine modules**, which is everything you have already built inside `mri/`. These are pure mathematical functions, and they have no idea that a GUI even exists.

Now you can see why the roadmap insisted on terminal-first development from the very beginning. Because your engine modules never import anything related to the GUI, the GUI ends up being a thin and entirely optional skin on top of them. That has a very practical consequence. Every bug you ever find can be reproduced in a five-line script, without clicking on anything, which makes it enormously easier to track down.

7.5 Implementation guide

What to Build — Step-by-Step (No Code, Just Instructions)

This milestone lives in `app.py`, and ideally you will also create a small `pipeline.py` that holds the coordinating engine class described above. For the GUI toolkit you have a choice. We recommend **PyQt6**, because it gives you professional threading support, which you are going to need. If you would prefer a gentler learning curve, CustomTkinter is a reasonable alternative.

A. The pipeline engine (build this before any window)

- Start by defining a simple settings container. It should list every parameter that the user is allowed to adjust, which means the trajectory type, the acceleration factor, the spoke count, the noise level, the window type and its width, and the checkbox flags.
- Next, write one method whose job is to take settings in and give results out. Internally that method selects a mask or a trajectory, applies the noise, subsamples or grids the data, filters it, reconstructs the image, and computes the metrics. Every one of those steps should be done by calling the modules you already wrote, so there is no new mathematics here.
- Test this method from a plain script before you build any window at all. Fabricate a settings object by hand, call the method, and print the metrics. While you are here, cache anything that can be reused across calls. For example, the full k -space of the input image does not change when only R changes, so when the user moves the R slider you should recompute the mask and not the FFT.

B. The static layout

- Build the main window using a grid layout. You want four embedded plot canvases arranged in a 2×2 grid, with a vertical control column on the right-hand side, exactly as in Figure 7.1.
- Now add the controls one by one. You need a dropdown for the trajectory, sliders for R , the spoke count, the noise level, and the window width, checkboxes for density compensation, log scaling, and the toggle between the error map and the PSF, and finally three read-only labels that will show the metrics.
- The first time you run the window, let all four panes show static placeholder images. Before you wire up any logic at all, resize the window and confirm that the layout

behaves sensibly.

C. Wiring events (single-threaded first)

- Connect the change event of every control to one shared callback. That callback reads all of the current settings, calls the pipeline engine, and then pushes the four result matrices into the panes and the three numbers into the labels.
- At this stage the interface will stutter whenever you drag a slider, and that is fine. We are getting the behaviour correct first, and we will make it smooth in the next step.
- Overlay the sampling mask on the k -space pane, for example by dimming the lines that were skipped. This is what lets the user see the cause and the effect sitting side by side.

D. Going asynchronous

- Move the pipeline call out of the callback and into a worker object that runs on a background thread. After this change, the callback's only job is to *launch* the worker with the current settings.
- Define a completion signal that carries the results bundle, and connect that signal to a method on the main thread. That method should do nothing except repaint the panes and update the labels.
- Add the busy-guard. The idea is simple. If a new event arrives while a worker is already running, you remember only the *latest* requested settings, and you run those once the worker is free. Everything that arrived in between gets dropped, because it is already out of date.
- Finally, go back and verify the two iron laws from earlier in this chapter. No widget should ever be touched from the worker thread, and dragging a slider rapidly should never build up a backlog of runs.

E. Comfort features (only after everything above works)

- Add a file-open dialog so the user can load their own images. Route the file through your Milestone 1 loader, so that grayscale conversion and resizing happen automatically without any extra work.
- Add a phantom button that restores the Shepp–Logan test image, so the user can always get back to a known starting point.
- Add a save-figure button that exports the current four-pane view to a PNG file. This is worth doing, because it is how you are going to generate every figure in your final report.

How to Test It Before Moving On

- **Responsiveness test:** drag the R slider rapidly back and forth. While you do this, the window must keep repainting and must never gray out, and within a moment of you releasing the slider it must settle on the correct final image.
- **Consistency test:** choose some specific settings in the GUI and note the metrics. Then run exactly the same settings through the pipeline engine in a plain script. The metrics must match exactly. This test is what proves that the GUI really is a pure skin over the engine.
- **Coherence test:** set a uniform mask with $R=2$. Three things must now be true at

the same time. Pane 2 must visibly show half of its lines dimmed, Pane 3 must show ghosting, and the PSNR label must drop. If any one of them is missing, something is out of sync.

- **Stale-frame test:** drag a slider quickly, stop, and watch how many updates the display goes through. It must jump straight to the final value. It must not replay every intermediate position for seconds after you have stopped moving.
- **Cross-feature test:** select the radial trajectory and then turn the density compensation checkbox off and on. The display must reproduce, live, the flip between glare and proper contrast that you saw in Chapter 5.

Self-Check: Can You Answer These?

1. What does “event-driven” mean, and how does it differ from a script that runs top to bottom?
2. Why does running the reconstruction inside the slider callback freeze the window? Explain using the receptionist analogy, then using the terms “main thread” and “event loop.”
3. State the two iron laws of GUI threading and what goes wrong when each is violated.
4. Why is it valuable that your engine modules never import GUI code? Name two concrete benefits.
5. What is a signal/slot connection, and why is it the *only* safe channel from worker to widgets?
6. A user drags the noise slider through 40 positions in one second. What should your application compute, and what should it skip?
7. Which computations are worth caching between slider events, and which must be redone every time?

The Build Plan — Order of Work, Edge Cases, and Polish

8.1 Why we build things in this exact order

Let us talk about the order in which you are going to build this project. We have laid the work out as a six-week sequence, and I want you to understand that this sequence was not chosen at random. The reason is that each stage you finish becomes the test harness for the stage that comes after it. When Week 1 works, it gives you a way to check Week 2. When Week 2 works, it gives you a way to check Week 3, and so on. Here is the whole plan in one table, with a short explanation of why each piece comes when it does:

Week	Build	Why now
1	Fourier engine (Ch. 2)	Every other part of the project either multiplies something into this engine or inverts something back out of it. Once its round-trip test passes, that test keeps protecting all of the later work.
2	Masks + PSF (Ch. 3)	This stage only needs an element-wise multiplication with the output you built in Week 1. It is also where the first dramatic artifacts appear on screen, which gives you strong motivation early on.
3	GUI skeleton (Ch. 7)	This may feel backwards, but the GUI in <code>app.py</code> comes <i>before</i> the hardest mathematics. Once you can watch sliders change images, Weeks 4 and 5 become something you can debug by eye, and the GUI learning is spread out across the weeks instead of piling up at the end.
4	Filters + noise (Ch. 4)	These are small, self-contained additions that you bolt onto an interactive loop that is already working.
5	Trajectories + gridding (Ch. 5)	This is the hardest milestone, so we attempt it only when the scaffolding around it (the visual feedback, the metrics, and the phantom) is mature and you can trust it.
6	Polish, tests, docs	Optimization and packaging only make sense once the behavior of the program is complete and no longer changing.

Now here is the single most important discipline in this whole plan, and I want you to take it seriously: **never start milestone $n+1$ while milestone n 's tests are still failing**. Every chapter in this guide ends with a test box, and you should treat that box as a gate you have to pass through rather than a friendly suggestion. The reason is simple. A bug that slips past its own milestone costs you about three times as much effort to find later, because by the time you notice it, three other modules in `mri/` have become possible suspects and you have to rule each of them out.

8.2 Edge cases you must handle (the Week 6 checklist)

What to Build — Step-by-Step (No Code, Just Instructions)

Input robustness

- **Non-square images:** when a user loads an image that is not square, you should zero-pad the shorter dimension until the image is square, and you should do this *before* resizing to 256×256 . If you resize first, the anatomy gets stretched. Padding with zeros is harmless from the point of view of k -space, because all it does is interpolate the spectrum.
- **Color inputs:** route every loaded image through the luminance conversion. You should never assume that an image arrives with a single channel.
- **Extreme value ranges:** think about what happens when someone loads an all-black input, or a constant image. These must not crash the normalizer, so guard the divide-by-max step. They must not crash the metrics either, so guard the case where the MSE is 0 inside the PSNR calculation.
- **Slider extremes:** when the user sets $R=1$, the program must reproduce the perfect round-trip. The maximum R , the minimum spoke count, and $\sigma_n=0$ must all behave sensibly as well, rather than throwing an error.

Performance (only where you have measured it, never where you merely suspect it)

- Profile first. Run the pipeline under a profiler and find out where the time actually goes before you change anything. In practice you will find that the gridding loop dominates, and that the FFTs almost never do.
- Vectorize the gridding spreader. Right now it is a Python loop that handles one sample at a time. Replace that with array operations that compute all four neighbor indices and their weights for *all* of the samples at once, and then deposit them using index-based accumulation.
- Cache the things that do not change. The full k -space of the input image does not change, the coordinate grids do not change, and each mask stays the same for a given set of parameters, so compute each of these once and reuse it.

Packaging and the report

- Organize the repository so that there is one folder for the engine modules in `mri/`, one folder for the tests, one for the GUI, a `README` that explains installation and usage, and a folder that holds your generated result figures.
- Turn every test box in this guide into an automated test file. This is easier than it sounds, because each box is already written as a set of pass/fail conditions.
- Notice that the report more or less writes itself from the experiments you have already run. You have the artifact-family comparison from Chapter 5, the PSNR-versus- R curves for each trajectory from Chapter 6, the density-compensation reveal, and the ringing-versus-window study from Chapter 4.
- Record the demo video last, and follow one narrative arc when you do. Start with full sampling, then accelerate, then let the ghosts appear, then switch strategies so the artifacts change character, and finally let the metrics quantify everything you just showed.

8.3 When you get stuck: the debugging ladder

Everyone gets stuck at some point in this project, so here is a list you can work down whenever that happens. In our experience it resolves the overwhelming majority of the problems you will run into:

1. **Do you see a checkerboard pattern over the image?** That almost always means an `fftshift` or `ifftshift` is missing somewhere. Go back and audit the sandwich rule from Chapter 2.
2. **Is the display completely black or completely white?** This is a scaling problem. Check whether you are displaying raw magnitudes without log compression, or whether you are comparing two images that live on different value ranges.
3. **Does the artifact run along the wrong axis?** You most likely have a transposed mask, or you have swapped the row and column indexing somewhere.
4. **Does the radial reconstruction look like glowing fog?** Your density compensation is either missing or it is not actually being multiplied in.
5. **Does the reconstruction look right while the metrics come out absurd?** This is a scale mismatch. One image is probably in $[0, 1]$ while the other is in $[0, 255]$.
6. **Does the GUI freeze?** You are doing the computation on the main thread. Go back and revisit Figure 7.2.

Reference: Glossary and Examiner's Question Bank

9.1 Glossary — every intimidating term, explained in plain words

k-space

This is the name we give to the image once it has been rewritten as a table of spatial frequencies. Each entry in that table records the strength of one striped wave pattern, which we call a grating, together with how far that pattern has been shifted sideways. Every chapter in this project starts from *k*-space, because that is what the scanner actually gives us.

Spatial frequency

This describes how rapidly the brightness goes up and down as you move across the picture. Wide, lazy stripes change slowly, so we call them low frequency. Tight, thin stripes change quickly, so we call them high frequency.

DC component

This is the entry at zero frequency, and after we shift *k*-space it sits right in the center of the matrix. Its value is equal to the sum of all the pixels, so you can think of it as the average brightness of the whole image.

FFT / IFFT

These are the fast algorithms that compute the discrete Fourier transform and its inverse. The forward one takes the image and gives you the recipe, and the inverse one takes the recipe and gives you the image back. Both steps are exact, so nothing is lost going in either direction.

fftshift / ifftshift

These two functions swap the quadrants of the matrix. The first one moves the DC term into the center so that the picture of *k*-space looks the way a human expects, and the second one moves it back again before we call the FFT.

Magnitude / Phase

Every entry in *k*-space is a complex number, and a complex number stores two facts. The magnitude tells you how strong that grating is, and the phase tells you how far sideways the grating has been shifted.

Nyquist criterion

This is the rule that tells you how often you must sample a signal to represent it faithfully. You need at least two samples per cycle of the highest frequency present. If you sample less often than that, you get aliasing.

Field of view (FOV)

This is the physical extent of the scene that an image is able to represent. It is inversely proportional to the spacing between the samples in *k*-space, which is why sparser samples give you a smaller field of view.

Acceleration factor *R*

This tells you what fraction of the *k*-space lines were actually acquired, written as a speed-up. When we say *R*=4, we mean that only one line in every four was kept.

Aliasing / ghosting / wrap-around

These three names all describe the same fold-over artifact. It appears when the effective field of view becomes smaller than the object, so shifted copies of the anatomy wrap around and overlap the image.

Sampling mask

This is a matrix made of ones and zeros that we multiply into k -space. It is our way of simulating which samples the scanner acquired and which ones it skipped.

Variable-density sampling

This is a mask that keeps the energy-rich center of k -space completely intact and only thins out the outer region. We use it because the center is where most of the image lives.

Point spread function (PSF)

This is the image that a single bright dot turns into after it passes through a sampling pattern. It matters so much because every artifact you will see is simply this PSF stamped onto every pixel of the picture.

Convolution theorem

This theorem says that multiplication in k -space is the same thing as convolution in image space, and the other way around as well. It is the single idea that explains ghosts, blur, ringing, and streaks all at once.

Gibbs ringing

These are the ripples that appear beside sharp edges. They are caused by cutting off the high frequencies abruptly instead of fading them out.

Apodization

This means multiplying k -space by a smooth window, such as a Hamming or a Gaussian window, so that the high frequencies fade away gradually. You trade the ringing for a mild amount of blur.

AWGN

This stands for additive white Gaussian noise. It is the random thermal crackle that gets added independently to the real channel and the imaginary channel of every single k -space sample.

Radial / spiral trajectory

These are paths through k -space that do not follow the usual Cartesian grid. A radial trajectory looks like the spokes of a wheel, and a spiral trajectory looks like a corkscrew. They give you robustness to motion and extra speed, but the samples no longer line up with the grid.

Density compensation function (DCF)

These are per-sample weights that correct for the fact that the crowded center of k -space is over-represented before we grid the data. For the radial case the weight is $w = |r|$, which is also known as the ramp filter.

Gridding

This is the step where we deposit the scattered trajectory samples onto the integer grid that the FFT demands. We can do it with nearest-neighbor assignment, with bilinear spreading, or with a kernel.

Streak artifact

This is what you get from radial undersampling instead of ghosting. You see straight lines fanning out across the image, and they come from the star-shaped PSF of a spoke pattern.

MSE / PSNR / SSIM

These are the three quality metrics we use. MSE is the average squared error per pixel.

PSNR is the same thing expressed on a decibel scale relative to the peak signal. SSIM is a perceptual score that compares local luminance, contrast, and structure between two images.

Parseval's theorem

This theorem tells you that the total energy is the same in image space and in k -space, up to a fixed constant. In other words, the Fourier transform loses nothing.

Shepp–Logan phantom

This is the standard synthetic test image that looks a little like a head. It is used throughout tomography and MRI research, and we use it throughout this project.

Event loop / callback

The event loop is the idle cycle in which a GUI sits and waits for the user to do something. A callback is the function that gets invoked when one of those events actually happens.

Worker thread / signal

A worker thread is a background lane where we put the heavy computation so it does not freeze the window. A signal is the thread-safe message that the worker sends back to the GUI when it is done.

9.2 The examiner's question bank

These are the questions an instructor is most likely to ask you at a project demonstration, and under each one I have written a model answer for you. You do not need to memorize these word for word. If you can explain each answer in your own words, then you are ready.

Foundations

Q: Why does MRI need the Fourier transform at all?

The scanner does not measure pixel brightnesses. What it physically measures are spatial frequencies, in the form of radio signals that have been encoded by magnetic gradients. So we end up with a table of frequencies, and we need a way to turn that table into an anatomical image. The inverse Fourier transform is exactly that mathematical bridge.

Q: What would the k -space of a perfectly uniform gray image look like?

You would see a single nonzero value at the DC center, and every other entry would be zero. That makes sense when you remember that a constant image contains only the zero-frequency grating, which is the “flat sheet” with no stripes at all.

Q: Why is k -space complex-valued?

Each grating needs two numbers to describe it. One is its amplitude, and the other is how far it has been shifted sideways. A complex number has a magnitude and a phase, and those two quantities store exactly those two facts.

Q: Where is most of the energy in a natural image's k -space, and why?

Most of the energy is concentrated in the low-frequency center. The reason is that natural images are mostly made of smooth regions, and the edges that need high frequencies are spatially rare.

Sampling and artifacts

Q: Can you derive what happens when you keep only every second k -space row?

When you keep every second row, you double the line spacing Δk . Since the field of view is given by $\text{FOV} = 1/\Delta k$, doubling the spacing halves the field of view. The object no longer fits inside that smaller field of view, so it wraps around, and you get a ghost shifted by

exactly half the image. You can reach the same answer another way. The PSF of this mask has two spikes that sit half an image apart, and convolution with that PSF stamps both of them onto the anatomy.

Q: Why are random-mask artifacts noise-like rather than ghost-like?

To get a clean ghost, the side energy of the PSF has to be concentrated into discrete spikes, and only a *periodic* mask produces spikes like that. A random mask has no rhythm to it, so its PSF spreads the side energy incoherently across the image. That is why the artifact ends up looking like grain instead of a copy.

Q: Suppose you increase R and the PSNR barely drops for VDS but plummets for uniform sampling. Why would that happen?

Variable-density sampling always keeps the fully sampled low-frequency center, and that center holds nearly all of the image energy and contrast. So when you raise R , the errors that survive can only touch the low-energy detail frequencies in the outer region, and the PSNR hardly notices.

Q: What is the PSF of a *full* sampling mask?

It is a single perfect spike. Convolution with a single spike changes nothing at all, and that is exactly why a fully sampled reconstruction is free of artifacts.

Filtering and noise

Q: Why can't Gibbs ringing be fixed by simply acquiring more frequencies?

If you truncate the frequencies at *any* finite radius, you get an overshoot of a fixed relative height next to an ideal edge, and that height is about 9%. Acquiring more frequencies does not make the overshoot smaller. It only squeezes the ripples closer to the edge. The only thing that actually removes them is a *smooth* taper.

Q: Can you show me informally why windowing blurs the image?

We start from the convolution theorem. Multiplying k -space by a window is the same as convolving the image with the inverse transform of that window. The inverse transform of a smooth window is a small, smooth blob. And when you convolve an image with a blob, you are blurring it by definition.

Q: Why do you add noise to k -space instead of adding it to the image?

We add it there because that is where the physics injects it. The antenna measures each complex sample through noisy electronics, so the noise arrives in the frequency data. There is a second benefit as well. After magnitude reconstruction, this approach reproduces the correct Rician noise floor in the dark regions, and noise added in the image domain would not do that.

Non-Cartesian imaging

Q: Tell me the two problems that radial data poses for FFT reconstruction, and the fix for each one.

The first problem is that the samples are not uniformly dense. The center of k -space is oversampled, and we fix that with the density-compensation weights $w = |r|$. The second problem is that the samples do not land on the integer grid. We fix that with gridding, which means interpolating or spreading the samples onto the grid.

Q: Why exactly is the radial weight proportional to $|r|$?

The spokes are spaced out at equal angles. That means the area each sample represents is a small annular wedge, and the area of that wedge grows linearly with the radius. When you weight each sample by $|r|$, you give every unit of k -space area equal representation again.

Q: Why do the four bilinear weights have to sum to one?

Each sample has to deposit exactly the signal that it measured onto the grid. It should not deposit more, and it should not deposit less. When the four weights sum to one, the total signal is conserved during the spreading step.

Q: Why do you get streaks instead of ghosts?

The artifact always takes the shape of the PSF. The inverse transform of a spoke pattern is star-shaped, so its side energy lies along straight lines, and those lines are the streaks. The PSF of a periodic row pattern, on the other hand, is a train of spikes, and those spikes are the ghosts.

Metrics and software

Q: Two reconstructions have identical MSE, but SSIM says one is far better than the other. What might they look like?

The first one might be mildly and uniformly blurred. Its structure is still preserved, so its SSIM is high. The second one might have spent the same error budget on localized ghosts or heavy noise. Its structure has been disrupted, so its SSIM is low.

Q: Why does your GUI stay responsive during heavy recomputation?

The reconstruction runs on a worker thread, so the main thread is free to keep servicing the event loop. When the worker finishes, it sends the results back through a thread-safe signal. The only work that runs on the main thread is the lightweight repaint.

Q: How do you know that your whole pipeline is correct?

We rely on several layers of evidence. For the transform core we have exact round-trip tests and Parseval tests. For the masks we compare the predicted ghost geometry with what we actually observe. For gridding we check weight conservation and run the dense-limit test. For the metrics we have identity and monotonicity tests. And for the application shell we check that the GUI and the script give consistent results.

Key Takeaways

You now hold the full map. There are five ideas, which are the matrix, the recipe, the damage, the re-gridding, and the measurement. There is one theorem that does most of the work, and that is convolution. There is one golden rule, which is the shift sandwich. And there is one discipline, which is to test every milestone before you move on to the next. Build it in that order, test at every gate, and the “overwhelming” roadmap becomes six calm weeks of matrix arithmetic. Good luck, and enjoy the moment when the first ghost crawls across your screen. You will know exactly why it is there.

Team Onboarding — Environment Setup and the Project Map

The Concept in Plain English

Let us start with why this chapter is here at all. We want any teammate to be able to sit down at a brand-new laptop and, within about fifteen minutes, have the app running and be editing code, without needing to ask anyone else for help. To get you there, the chapter is split into two halves. The first half is the **environment setup**. It gives you the exact commands to run, in the order you should run them, together with the failures you are most likely to bump into along the way. The second half is the **project map**. That is a guided tour of every file and folder in the repository. For each one we will talk about what lives inside it and which milestone it belongs to. We will also spend some time on the single architectural rule that keeps this codebase from collapsing into a tangle, because it is the one thing we really need everyone to respect.

10.1 What must already be on your machine

Before we run any commands, there are three things that need to be installed on your machine. Let us go through them one at a time.

- **Python 3.12 (64-bit)**. Please make sure it is really 3.12 and not 3.13. In particular, stay away from the “free-threaded” build that is labelled 3.13t. The reason is that two of our packages, `PySide6` and `scipy`, do not yet ship reliable wheels for that build, so the install will either fail or behave strangely. You can check which versions you have by running `py --list`. If 3.12 is not in that list, install it from python.org, and while the installer is running, remember to tick the box that says *Add python.exe to PATH*.
- **Git**. We use Git to clone the repository onto your machine and to push your changes back up for the rest of the team.
- **VS Code** with the Microsoft *Python* extension installed. You are welcome to use any editor you prefer, but please be aware that the steps below assume you are in VS Code, so you may have to translate a little if you use something else.

10.2 Setup, step by step

A quick note before we begin. Every command in this section is written for **PowerShell**, and every command expects you to be sitting in the project root. The project root is simply the folder that contains `app.py`. If you are ever unsure whether you are in the right place, look for that file.

Step 1 — Get the code

The first thing we need is a copy of the code on your machine. These two commands clone the repository and then move you into the folder it created.

```
git clone <repository-url>
cd log_synthesis
```

Step 2 — Create the virtual environment, pinned to 3.12

Next we create a virtual environment, and we tell Python explicitly that we want it built on version 3.12. The `-3.12` part of the command is what does that pinning, so please do not leave it out.

```
py -3.12 -m venv .venv
```

Why This Matters

You may be wondering what a *virtual environment* actually is. The simplest way to think about it is that it is a private copy of Python that lives inside the project folder. When you install a package into this private copy, that package cannot interfere with any other project on your machine. It also means that everybody on the team ends up with exactly the same package versions, which matters a great deal when we are comparing results. And if something goes badly wrong, you can simply delete the `.venv` folder and all of it is undone. This is the standard way Python teams avoid the classic “it works on my machine” conversation.

Step 3 — Activate it

Creating the environment is not quite enough. We also have to switch it on, and that is what activation does.

```
.\.venv\Scripts\Activate.ps1
```

Once this has worked, your prompt should begin with `(.venv)`. That little prefix is how you know the environment is active. There are two failures that people run into here, so let us look at both.

- If PowerShell tells you “*running scripts is disabled on this system*”, then Windows is blocking the activation script as a safety measure. The fix is to run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first. That command only affects the current window, so it is safe. After that, try the activation command again.
- If you run `python --version` and it prints something other than 3.12.x, then the activation did not take, even if it looked like it worked. The easiest fix is to open a fresh terminal and redo this step from the beginning.

One thing to burn into your habits: every time you open a fresh terminal window, you have to activate the environment again before you run anything. The environment does not stay switched on between sessions. Almost every time a teammate comes to you saying “but I installed numpy, why is it telling me `ModuleNotFoundError`?”, the answer is that they opened a new terminal and forgot this step.

Step 4 — Install the dependencies

Now that the environment is active, we can install the packages the project depends on. The first command brings `pip` itself up to date, and the second reads the list of packages from `requirements.txt` and installs every one of them.

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

There are five packages in that file, and it helps to know what each one is doing for us. Here is the list along with the job each package has in the project.

Package	What it is for
<code>numpy</code>	all matrix math and the FFT (Milestone 1)
<code>scipy</code>	scattered-point interpolation for gridding (Milestone 5)
<code>scikit-image</code>	Shepp–Logan phantom and the SSIM metric
<code>matplotlib</code>	drawing images into the four GUI panels
<code>PySide6</code>	the desktop window, sliders, and threading

At some point you may find you need a package that is not on this list. That is perfectly fine, but as soon as you install it, please regenerate the manifest with the command below. The `requirements.txt` file is how the rest of the team finds out about your new package, so if you skip this step their copy of the project will break the next time they pull your code.

```
python -m pip freeze > requirements.txt
```

Step 5 — Point VS Code at the `venv`

The terminal now knows about your environment, but VS Code does not yet. To fix that, open the command palette with `Ctrl+Shift+P`, run *Python: Select Interpreter*, and pick the entry that shows `.venv\Scripts\python.exe`. It is worth understanding that this is a separate thing from activating the environment in the terminal. Activating the terminal is what lets your scripts run. Selecting the interpreter in VS Code is what gives you autocomplete for `numpy` and `PySide6`, and it is also what puts red squiggles under your typos before you ever run the code.

Step 6 — Smoke test

Before you write a single line of real code, let us prove that the whole environment works. Create a throwaway script inside `scratch/` and have it do the following things.

- First, import `numpy`, `scipy.interpolate`, `skimage`, `matplotlib` (using the `QtAgg` backend), and `PySide6.QtWidgets`. If any of these imports fails, you know straight away which package is missing.
- Next, load the Shepp–Logan phantom and print its shape. This shows that `scikit-image` is installed and working.
- Then run one FFT followed by an inverse FFT and print the maximum error between the result and what you started with. You should expect something around $\sim 10^{-13}$, which is essentially zero.
- Finally, open a bare Qt window, with nothing in it.

If the window opens and the error is tiny, then every single layer of the stack is working, from the raw math library all the way up to the desktop toolkit, and you are ready to go.

10.3 The project map

Now let us take a tour of the repository so that you know where everything lives. Here is the folder layout, with a short note beside each entry.

```
log_synthesis/  
|- .venv/          your private Python -- never committed  
|- docs/          the roadmap + this guide (docs/guide/ holds the LaTeX)
```

```

|- mri/                THE ENGINE: pure math, no GUI imports, ever
|  |- __init__.py      empty; makes 'mri' importable as a package
|  |- kspace_core.py   Milestone 1: FFT <-> image round trip
|  |- cartesian_sampling.py Milestone 2: the four binary masks
|  |- filters.py       Milestone 3: apodization windows + noise
|  |- trajectories.py  Milestone 4-5: radial/spiral coordinates
|  |- gridding.py      Milestone 5: non-Cartesian -> Cartesian grid
|  |- metrics.py       MSE / PSNR / SSIM
|  |- pipeline.py      the coordinator the GUI calls
|- scratch/            throwaway experiments and saved plots -- gitignored
|- app.py              the PySide6 window (Milestone 6). The ONLY file
|                      that imports Qt.
|- requirements.txt    exact package versions -- committed
|- .gitignore          excludes .venv, __pycache__, scratch, LaTeX debris

```

Why This Matters

The one architectural rule. If you remember nothing else from this chapter, please remember this. The `mri/` folder is the engine, and `app.py` is the face that the user sees. The dependency between them points in exactly one direction. `app.py` is allowed to import things from `mri`. Nothing inside `mri` is ever allowed to import `PySide6` or anything else related to `Qt`. Let us talk about why we care so much. Because the engine knows nothing about the GUI, you can test every milestone from a plain terminal script, without opening a window at all. It also means two people can work on the engine and the GUI at the same time without stepping on each other and creating merge conflicts. And this same separation is the foundation that Chapter 7 builds its whole architecture on. So if you ever catch yourself typing `from PySide6 import ...` inside the `mri` folder, please stop, because you are about to break the project.

A few practical conventions fall naturally out of that map, and it is worth going through them so that nothing surprises you later.

- **Milestone test scripts**, such as `test_week1_math.py`, live at the project *root*, right next to `app.py`. We put them there for a simple reason. When a script sits at the root, a line like `from mri.kspace_core import ...` just works, with no path tricks needed. To run one, make sure the `venv` is active, stay in the root folder, and type `python test_week1_math.py`.
- `scratch/` is gitignored on purpose. This is the place to dump experiments, saved figures, and half-formed ideas, and you should feel no guilt about leaving a mess there. When something in `scratch/` turns out to be worth keeping, it graduates into `mri/` or `docs/`.
- **Never commit** `.venv/` or `__pycache__/`. Those folders are specific to your machine and would only clutter the repository. Instead, the file we commit is `requirements.txt`, because that is the portable form of your environment.
- **Who edits what.** Because the layout gives each milestone its own file, each teammate can own whole files rather than sharing them with someone else. That is the cheapest merge-conflict insurance we could possibly have.

How to Test It Before Moving On

How do you know you are done with onboarding? You are finished when all four of the following are true.

- When you run `python --version` inside the activated `venv`, it prints 3.12.x.

- The smoke-test window opens, and the round-trip error it prints is around $\sim 10^{-13}$.
- When you type `np.fft.` in VS Code, it offers you autocomplete suggestions.
- When you run `git status`, it does *not* list `.venv/` or `scratch/` as untracked.

PySide6 Field Guide — Just Enough Qt, in Plain English

The Concept in Plain English

Back in Chapter 7 we designed the architecture of the application, meaning we decided which pieces exist and how they talk to each other. This chapter is different. Think of it as the *vocabulary lesson* that lets you actually speak Qt when you sit down to write `app.py`, which is the file that holds our GUI. PySide6 is the official Python binding for **Qt**. Qt itself is a C++ framework, and it is the same framework behind programs you may already know, such as VLC, OBS Studio, Telegram Desktop, and Maya. Now, Qt is enormous, and I want to reassure you right away that you do not need most of it. The whole simulator uses perhaps a dozen classes. This chapter introduces exactly those classes. It also explains the three ideas that make Qt feel alien the first time you meet it, namely the event loop, layouts, and signals. At the end we will go through the styling recipe and the traps that catch almost every beginner.

11.1 Idea 1: the event loop — your program becomes a waiter

Let us start with the biggest mental shift. A terminal script, the kind you have probably written many times, runs from the top of the file to the bottom and then exits. A GUI program does not behave like that. It starts up, and then it simply *waits*. It waits for clicks, for key presses, for slider drags, and every time one of those arrives it reacts, and then it goes back to waiting. That cycle of waiting and reacting is what we call the **event loop**.

Analogy

Here is the picture I want you to keep in your head. Think of a restaurant waiter. The waiter does not cook anything on their own schedule. Instead they stand ready, and every request from a customer, which in our vocabulary is an *event*, triggers a short burst of work. Now imagine one request takes twenty minutes for the waiter to handle personally. During those twenty minutes every other table is ignored. That is exactly how a frozen GUI happens, and it was the central problem we worried about in Chapter 7. Some handler hogged the waiter, and so the whole restaurant stopped.

So what does this look like in practice, in our `app.py`?

- Every Qt application creates exactly one `QApplication` object at startup. This object is the thing that owns the event loop, so you can think of it as the waiter being hired.
- You then build your window, call its `show()` method so it appears on screen, and hand control over to Qt by calling `app.exec()`. I want you to notice something important about that call: it *blocks until the user closes the window*. From that moment on, all of your logic runs inside event handlers, because the waiter is in charge now.
- Here is the corollary that trips people up. Any code you write after the line `app.exec()` only runs at shutdown, once the window is already gone. Beginners constantly put setup code there and then wonder why it never seems to run. Now you know why.

11.2 Idea 2: widgets and layouts — Lego bricks that resize themselves

Everything you can see in a Qt application is a **widget**. A widget is simply a visible building block, and widgets nest inside one another. The window contains panels, and the panels contain sliders, and so on down. The table below lists the dozen or so widgets that this project actually needs, so you do not have to go hunting through the documentation.

Class	What it is, in one line
QMainWindow	the outer application window; it owns a central widget and a status bar
QWidget	a blank rectangle, which is the generic container you build panels out of
QLabel	a piece of text (or an image), which is what your metric readouts are
QSlider	a draggable slider that emits its value as it moves
QComboBox	a dropdown, used for your mask-type and trajectory selectors
QPushButton	a clickable button, such as “Load Image” or “Reset”
QCheckBox	an on/off toggle, such as “Apply apodization”
QGroupBox	a titled frame that visually groups related controls together
QStatusBar	the strip along the bottom of the window for short, temporary messages
QFileDialog	the native “Open File...” dialog, available through one static call

Here is a rule I want you to take seriously from day one: **do not place widgets by typing in pixel coordinates**. It feels natural at first, but the moment the user resizes the window, or runs the app on a screen with a different resolution, everything you placed by hand is in the wrong spot. Qt’s answer to this problem is something called the **layout manager**. The idea is that you drop your widgets into a layout, and the layout computes where they go, and then it recomputes those positions on every resize without you lifting a finger. Once you get used to it you will not want to go back.

- `QHBoxLayout` arranges its children in a horizontal row, side by side.
- `QVBoxLayout` arranges its children in a vertical column, one above the other.
- `QGridLayout` arranges its children in rows and columns. This is what you will use for the 2×2 grid of image panels.
- Layouts nest freely inside one another. Our main window is one horizontal layout that holds a vertical column of controls on one side and the grid of four panels beside it. If you look back at the sketch in Chapter 7, that is exactly the picture we drew there.
- There is one more concept here called *stretch factors*. They decide who gets the spare space when the window grows. If you give the image grid a large stretch and give the control column none, then resizing the window enlarges the images while the sliders stay exactly where they are. That is usually what you want.

11.3 Idea 3: signals and slots — widgets announce, they never command

At this point almost every beginner asks the same question, and it is a good one: “how does my slider actually *call* my reconstruction function?” Qt’s answer is deliberately indirect, and I want to explain why that indirection is a good thing rather than an annoyance.

Analogy

Think of a widget as a radio station rather than a telephone caller. When a slider moves, it does not reach into your code and call anything. Instead it *broadcasts* an announcement to whoever is listening, something like “my value is now 4!” In Qt that announcement is called a **signal**. Any function may tune in to that broadcast by *connecting* to the signal, and a function that has connected is called a **slot**. One signal may feed many slots. One slot may listen to many signals. And here is the important part: neither side knows that the other one exists. That ignorance is the whole feature. Your engine code inside `mri/` never mentions sliders, which means it stays testable without a GUI anywhere in sight.

So what do you actually write in `app.py`? Let me walk you through it.

- Every widget ships with ready-made signals, so you rarely have to invent any. Sliders emit `valueChanged`, buttons emit `clicked`, and dropdowns emit `currentIndexChanged`.
- Connecting a signal to your function takes exactly one line. You name the widget, then the signal, then call its `connect` method with your function as the argument, so the shape is always `widget.signal.connect(function)`. You do this once, at the time the window is constructed, and then you forget about it.
- When a signal fires, it passes its payload along as an argument to the slot. For a slider that payload is the new value, and for a dropdown it is the chosen index.
- You can also declare *custom* signals on your own classes. This is precisely how the background worker from Chapter 7 reports “reconstruction finished” back to the window. The reason we use a signal for this rather than anything else is that signals are the **only** thread-safe bridge in Qt.
- One practical detail: `QSlider` only speaks integers. Suppose you want a continuous quantity like the noise level $\sigma \in [0, 0.2]$. The trick is to let the slider run from 0 to 200 and then divide by 1000 inside the slot. This is a standard Qt idiom and you will see it everywhere.

11.4 Embedding matplotlib: the canvas is just another widget

Now let us talk about the four image panels. They are matplotlib figures, but they live inside the Qt window rather than in their own pop-up. The bridge that makes this possible is a single class called `FigureCanvasQTAgg`, which you import from `matplotlib.backends.backend_qtagg`. The key thing to understand about it is that it *is* a `QWidget`. Because of that, it drops into a layout just like any label or button would. Here are the working rules you should follow.

- Create one `Figure` and one canvas per panel, and create them **once** at startup. Then, on every update, reuse the existing image object by calling `set_data` on it, rather than clearing and redrawing the axes. I stress this because rebuilding the axes on every slider tick is the classic cause of a sluggish GUI.
- When you want the screen to repaint, ask for it with `draw_idle()` rather than `draw()`. The difference is that `draw_idle()` coalesces a burst of update requests into a single paint, and that matters a great deal while a slider is being dragged.

- Remember that these panels show images, not plots. So switch the axes off, and set the figure face color to match the window background. If you skip this, the panels will look like they are floating on white rectangles.
- Import matplotlib *after* you have selected the QtAgg backend, and never call `plt.show()` inside the app. The reason is that the Qt window owns the screen now, and pyplot does not.

11.5 Keeping it responsive: the two-line summary of Chapter 7

The full design of the worker thread lives in Chapter 7, so I will not repeat it here. What I want to give you is just the Qt mechanics that make it work.

- The heavy math runs inside a worker object that has been moved onto a `QThread`. When the worker finishes, the results come back through a custom signal that carries the finished arrays.
- **Only the main thread may touch widgets.** The worker computes and emits a signal, and then the window's slot does the repainting. There are no exceptions to this rule, and I will say that again in the traps table below.
- Debounce your sliders with a single-shot `QTimer` set to roughly ~ 150 ms. Every movement of the slider restarts the timer, and only the timer's timeout actually triggers a recomputation. The effect is that dragging the slider fires one job instead of fifty.

11.6 Making it look good: the two-step styling recipe

I will be honest with you: out of the box, Qt looks a little dated. The good news is that two cheap steps fix it.

- **Step 1 — Fusion + palette.** At startup, select the built-in style called `Fusion`. It looks identical on every operating system, and it was designed to be recolored. Then hand your `QApplication` a `QPalette` that defines your background, surface, text, and one accent color. This is about twenty lines of code, and it transforms the whole app.
- **Step 2 — Qt Style Sheets (QSS).** QSS is a dialect of CSS made for widgets. You write selectors such as `QSlider::handle` or `QGroupBox::title`, and then you attach properties for padding, border-radius, and hover states. Use it for rounded control panels, a monospace font on the metric readouts, and slider styling. You apply the whole sheet once by calling `app.setStyleSheet(...)`.
- Please remember the warning from Chapter 7: Qt theming does *not* touch the matplotlib canvases. You have to style those from the matplotlib side, which means setting the figure face color, turning off the axes, and picking a good colormap for the error heatmap. If you forget, the app will look like plots bolted into a window.
- Skip QML and Qt Quick entirely for this project. They are a second language to learn on top of everything else, and matplotlib embeds poorly in them anyway.

11.7 The five classic beginner traps

Symptom	Cause and cure
Window flashes open and instantly vanishes	You kept no reference to it, so Python garbage-collected your window. Store it in a variable that outlives the <code>show()</code> call.
GUI freezes during recomputation	Heavy math ran in an event handler on the main thread. Move it to the worker described in Chapter 7.
Crash: “cannot create children for a parent in a different thread” Nothing happens when the slider moves	A worker touched a widget. Workers should only emit signals, and only the main thread should paint. Either the <code>connect</code> call was never made, or you connected the signal to the <i>result</i> of calling the function (with parentheses) instead of to the function itself.
Two windows appear, or matplotlib opens its own	There is a stray <code>plt.show()</code> or a second <code>QApplication</code> somewhere. You want one application object, one <code>exec()</code> , and no pyplot windows.

Key Takeaways

- There is one `QApplication` and one event loop. Once you call `app.exec()`, all of your code lives in slots.
- Widgets go into layouts, and layouts nest inside other layouts. Positioning by pixels is forbidden.
- Widgets broadcast signals, and your functions subscribe to them as slots. The two sides never need to know about each other.
- The matplotlib canvas is just another widget. Create it once, and then update it with `set_data` followed by `draw_idle`.
- Do the computing on the worker thread, do the painting on the main thread, bridge the two with signals, and debounce the sliders with a timer.
- Fusion, plus a palette, plus a little QSS, plus styled matplotlib canvases, adds up to an app that looks deliberate.

Self-Check: Can You Answer These?

1. What is the event loop, and why does a long computation inside a slot freeze the entire window?
2. Explain signals and slots to a teammate using the radio-station analogy. Why does this indirection keep the engine GUI-free?
3. Why must widget updates happen only on the main thread, and what is the sanctioned mechanism for a worker to deliver results?
4. Your slider must control $\sigma \in [0, 0.2]$ but `QSlider` is integer-only. What is the standard idiom for handling this?
5. Why is `draw_idle()` preferred over `draw()` when a slider is being dragged?
6. Name the two styling steps that take a default Qt app to a polished-looking one, and the part of the window they *cannot* style.